

Software Project Planning



Software Project Planning

After the finalization of SRS, we would like to estimate size, cost and development time of the project. Also, in many cases, customer may like to know the cost and development time even prior to finalization of the SRS.

Software Project Planning

In order to conduct a successful software project, we must understand:

- Scope of work to be done
- The risk to be incurred
- The resources required
- The task to be accomplished
- The cost to be expended
- The schedule to be followed

Software Project Planning

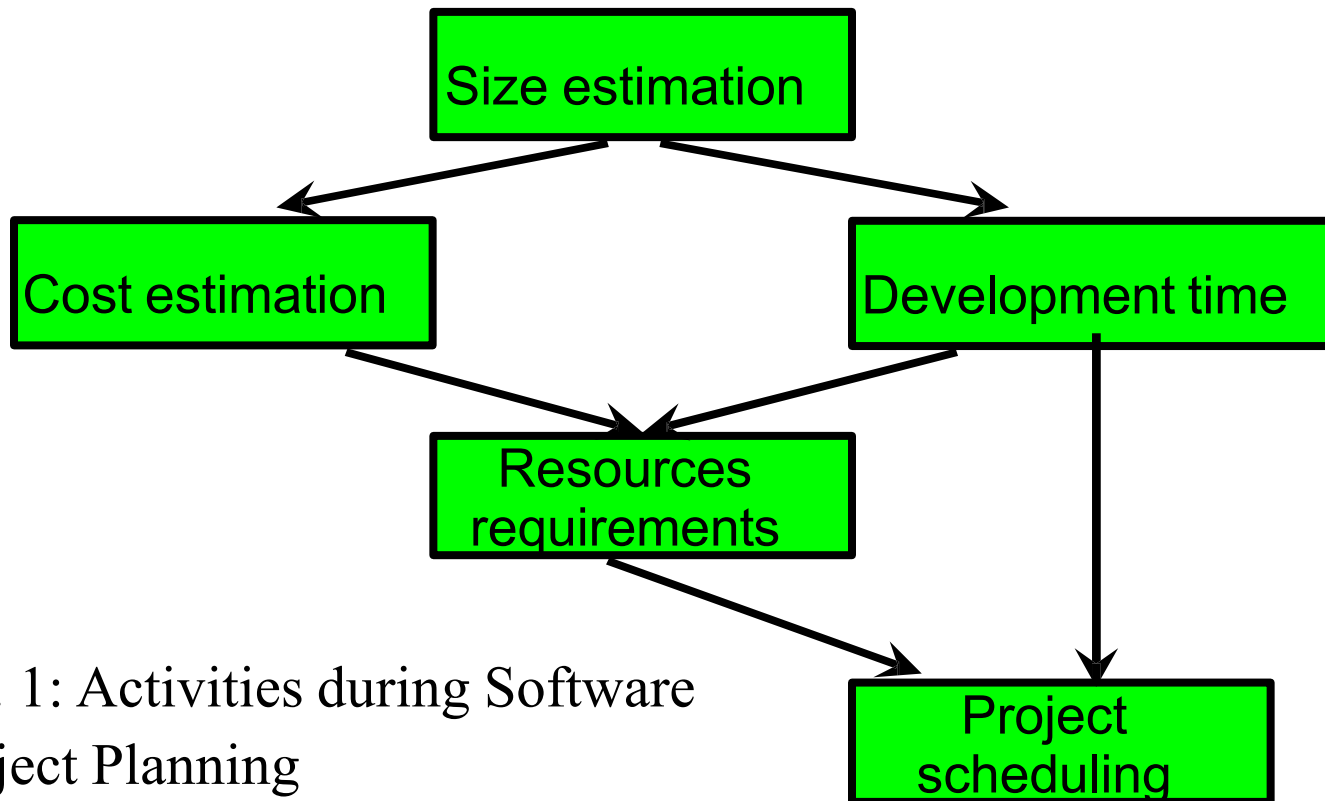


Fig. 1: Activities during Software Project Planning

- First Activity is to estimate the size of the project. The Size is the key parameter for the estimation of other activities.
- It is an input to all costing models for the estimation of cost, development time and schedule for the project.
- Resource requirements are estimated on the basis of cost and development time.

Software Project Planning

(i) Size Estimation (size oriented metrics)

Lines of Code (LOC)

If LOC is simply a count of the number of lines then figure shown below contains 18 LOC .

When comments and blank lines are ignored, the program in figure 2 shown below contains 17 LOC.

Fig. 2: Function for sorting an array

1.	int. sort (int x[], int n)
2.	{
3.	int i, j, save, im1;
4.	/*This function sorts array x in ascending order */
5.	If (n<2) return 1;
6.	for (i=2; i<=n; i++)
7.	{
8.	im1=i-1;
9.	for (j=1; j<=im; j++)
10.	if (x[i] < x[j])
11.	{
12.	Save = x[i];
13.	x[i] = x[j];
14.	x[j] = save;
15.	}
16.	}
17.	return 0;
18.	}

Software Project Planning

“A line of code is any line of program text that is not a comment or blank line, regardless of the number of statements or fragments of statements on the line. This specifically includes all lines containing program header, declaration, and executable and non-executable statements”.

This is the predominant definition for lines of code used by researchers. By this definition, figure shown above has 17 LOC.

- **Advantages of Size Oriented Metrics**

- LOC can be easily counted
- Many software estimation models use LOC or KLOC as input.

- **Disadvantages of Size Oriented Metrics**

- LOC measures are language dependent, programmer dependent
- Their use in estimation requires a lot of detail which can be difficult to achieve.

Software Project Planning

(ii) Function oriented metrics:

Alan Albrecht while working for IBM, recognized the problem in size measurement in the 1970s, and developed a technique (which he called Function Point Analysis), which appeared to be a solution to the size measurement problem.

Function Point (FP): FP measures functionality from the users point of view, that is, on the basis of what the user requests and receives in return from the system.

Software Project Planning

The principle of Albrecht's function point analysis (FPA) is that a system is decomposed into functional units.

- Inputs : information entering the system
- Outputs : information leaving the system
- Enquiries : requests for instant access to information
- Internal logical files : information held within the system
- External interface files : information held by other system that is used by the system being analyzed.

Software Project Planning

The FPA functional units are shown in figure given below:

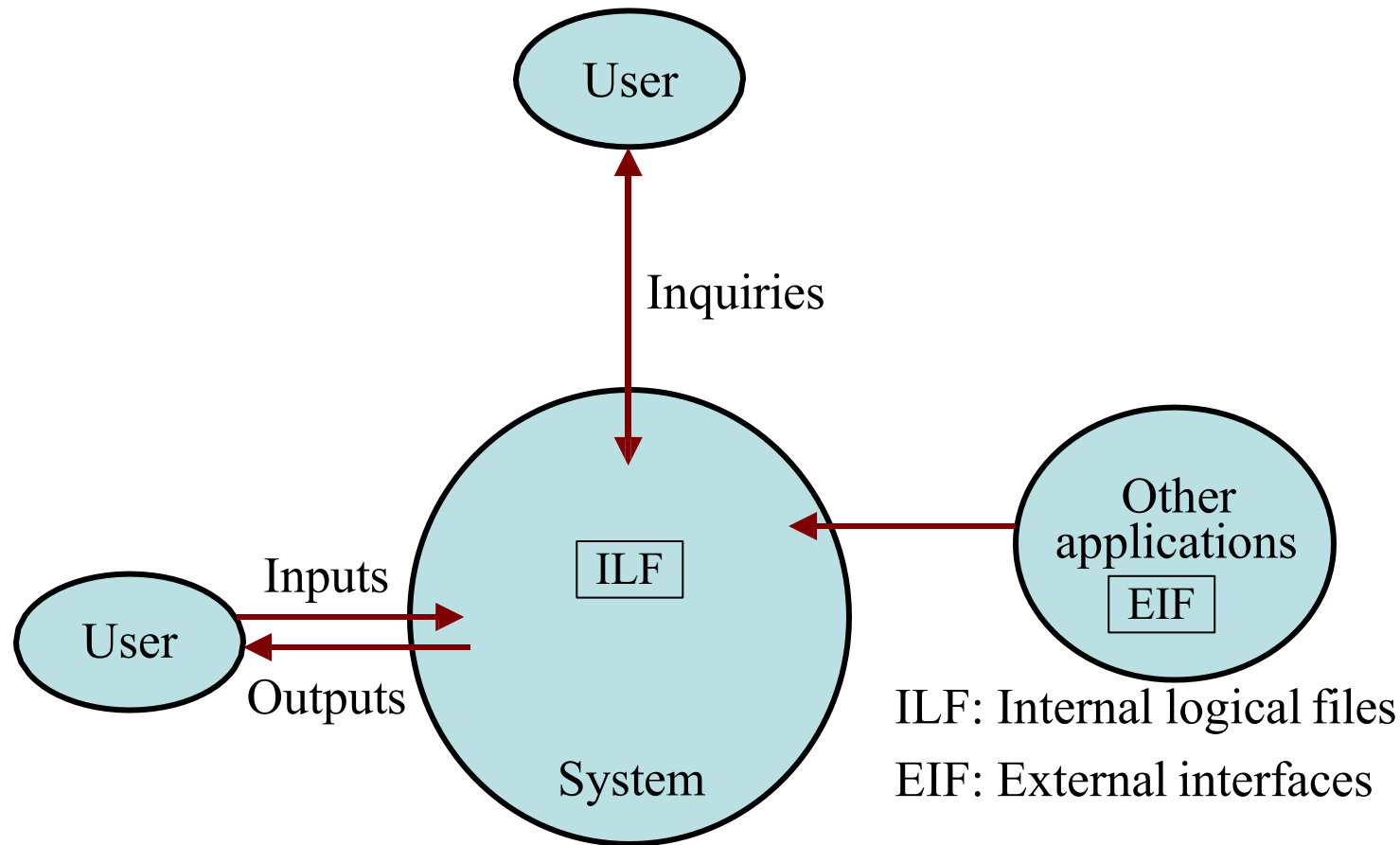


Fig. 3: FPAs functional units System

Software Project Planning

The five functional units are divided in two categories:

(i) Data function types

- Internal Logical Files (ILF): A user identifiable group of logical related data or control information maintained within the system.
- External Interface files (EIF): A user identifiable group of logically related data or control information referenced by the system, but maintained within another system. This means that EIF counted for one system, may be an ILF in another system.

Software Project Planning

(ii) Transactional function types

- External Input (EI): An EI processes data or control information that comes from outside the system. The EI is an elementary process, which is the smallest unit of activity that is meaningful to the end user in the business.
- External Output (EO): An EO is an elementary process that generate data or control information to be sent outside the system.
- External Inquiry (EQ): An EQ is an elementary process that is made up to an input-output combination that results in data retrieval.

Software Project Planning

Special features

- Function point approach is independent of the language, tools, or methodologies used for implementation; i.e. they do not take into consideration programming languages, data base management systems, processing hardware or any other data base technology.
- Function points can be estimated from requirement specification or design specification, thus making it possible to estimate development efforts in early phases of development.

Software Project Planning

- Function points are directly linked to the statement of requirements; any change of requirements can easily be followed by a re-estimate.
- Function points are based on the system user's external view of the system, non-technical users of the software system have a better understanding of what function points are measuring.

Software Project Planning

After classifying each of the five function type the unadjusted function point (UFP) are calculated using predefined weights for each function type as given in table.

Counting function points

Functional Units	Weighting factors		
	Low	Average	High
External Inputs (EI)	3	4	6
External Output (EO)	4	5	7
External Inquiries (EQ)	3	4	6
Internal logical files (ILF)	7	10	15
External Interface files (EIF)	5	7	10

Table 1 : Functional units with weighting factors

Software Project Planning

Table 2: UFP calculation table

Functional Units	Count	Complexity	Complexity Totals	Functional Unit Totals
External Inputs (EIs)		Low x 3	=	
		Average x 4	=	
		High x 6	=	
External Outputs (EOs)		Low x 4	=	
		Average x 5	=	
		High x 7	=	
External Inquiries (EQs)		Low x 3	=	
		Average x 4	=	
		High x 6	=	
Internal logical Files (ILFs)		Low x 7	=	
		Average x 10	=	
		High x 15	=	
External Interface Files (EIFs)		Low x 5	=	
		Average x 7	=	
		High x 10	=	
Total Unadjusted Function Point Count				

Software Project Planning

The weighting factors are identified for all functional units and multiplied with the functional units accordingly. The procedure for the calculation of Unadjusted Function Point (UFP) is given in table shown above.

Software Project Planning

The procedure for the calculation of UFP in mathematical form is given below:

$$UFP = \sum_{i=1}^5 \sum_{j=1}^3 Z_{ij} w_{ij}$$

Where i indicate the row and j indicates the column of Table 1

w_{ij} : It is the entry of the i^{th} row and j^{th} column of the table 1

Z_{ij} : It is the count of the number of functional units of Type i that have been classified as having the complexity corresponding to column j .

Software Project Planning

Organizations that use function point methods develop a criterion for determining whether a particular entry is Low, Average or High. Nonetheless, the determination of complexity is somewhat subjective.

$$FP = UFP * CAF$$

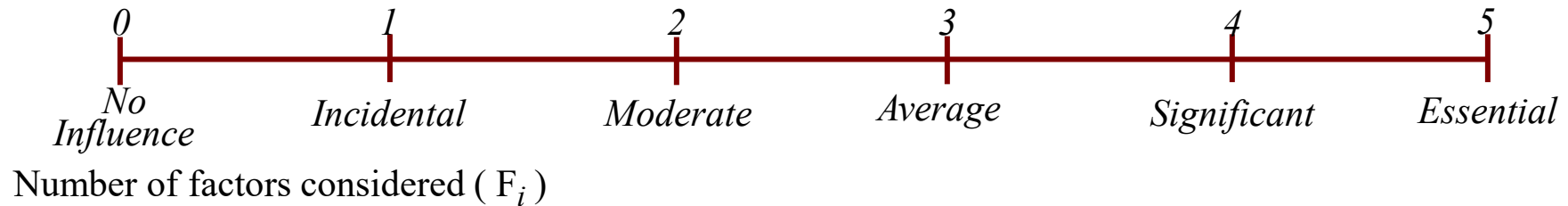
Where CAF is complexity adjustment factor and is equal to $[0.65 + 0.01 \times \sum F_i]$.

The F_i ($i=1$ to 14) are the degree of influence and are based on responses to questions noted in table 3.

Software Project Planning

Table 3 : Computing function points.

Rate each factor on a scale of 0 to 5.



1. Does the system require reliable backup and recovery ?
2. Is data communication required ?
3. Are there distributed processing functions ?
4. Is performance critical ?
5. Will the system run in an existing heavily utilized operational environment ?
6. Does the system require on line data entry ?
7. Does the on line data entry require the input transaction to be built over multiple screens or operations ?
8. Are the master files updated on line ?
9. Is the inputs, outputs, files, or inquiries complex ?
10. Is the internal processing complex ?
11. Is the code designed to be reusable ?
12. Are conversion and installation included in the design ?
13. Is the system designed for multiple installations in different organizations ?
14. Is the application designed to facilitate change and ease of use by the user ?

Software Project Planning

Example: 4.1

Consider a project with the following functional units:

Number of user inputs = 50

Number of user outputs = 40

Number of user enquiries = 35

Number of user files = 06

Number of external interfaces = 04

Assume all complexity adjustment factors and weighting factors are average.

Compute the function points (FP) for the project.

Software Project Planning

Solution

We know

$$UFP = \sum_{i=1}^5 \sum_{J=1}^3 Z_{ij} w_{ij}$$

$$\begin{aligned} UFP &= 50 \times 4 + 40 \times 5 + 35 \times 4 + 6 \times 10 + 4 \times 7 \\ &= 200 + 200 + 140 + 60 + 28 = 628 \end{aligned}$$

$$\begin{aligned} CAF &= (0.65 + 0.01 \sum F_i) \\ &= (0.65 + 0.01 (14 \times 3)) = 0.65 + 0.42 = 1.07 \end{aligned}$$

$$\begin{aligned} FP &= UFP \times CAF \\ &= 628 \times 1.07 = 672 \end{aligned}$$

Software Project Planning

Example:4.2

An application has the following:

10 low external inputs, 12 high external outputs, 20 low internal logical files, 15 high external interface files, 12 average external inquiries, and a value of complexity adjustment factor of 1.10.

What are the unadjusted and adjusted function point counts ?

Software Project Planning

Solution

Unadjusted function point counts may be calculated using as:

$$UFP = \sum_{i=1}^5 \sum_{J=1}^3 Z_{ij} w_{ij}$$
$$= 10 \times 3 + 12 \times 7 + 20 \times 7 + 15 \times 10 + 12 \times 4$$
$$= 30 + 84 + 140 + 150 + 48$$
$$= 452$$

$$FP = UFP \times CAF$$
$$= 452 \times 1.10 = 497.2.$$

Software Project Planning

Example: 4.3

Consider a project with the following parameters.

- (i) External Inputs:
 - (a) 10 with low complexity
 - (b) 15 with average complexity
 - (c) 17 with high complexity
- (ii) External Outputs:
 - (a) 6 with low complexity
 - (b) 13 with high complexity
- (iii) External Inquiries:
 - (a) 3 with low complexity
 - (b) 4 with average complexity
 - (c) 2 high complexity

Software Project Planning

- (iv) Internal logical files:
 - (a) 2 with average complexity
 - (b) 1 with high complexity
- (v) External Interface files:
 - (a) 9 with low complexity

In addition to above, system requires

- i. Significant data communication
- ii. Performance is very critical
- iii. Designed code may be moderately reusable
- iv. System is not designed for multiple installation in different organizations.

Other complexity adjustment factors are treated as average. Compute the function points for the project.

Software Project Planning

Solution: Unadjusted function points may be counted using table 2

Functional Units	Count	Complexity		Complexity Totals	Functional Unit Totals
External Inputs (EIs)	10	Low x 3	=	30	192
	15	Average x 4	=	60	
	17	High x 6	=	102	
External Outputs (EOs)	6	Low x 4	=	24	115
	0	Average x 5	=	0	
	13	High x 7	=	91	
External Inquiries (EQs)	3	Low x 3	=	9	37
	4	Average x 4	=	16	
	2	High x 6	=	12	
External logical Files (ILFs)	0	Low x 7	=	0	35
	2	Average x 10	=	20	
	1	High x 15	=	15	
External Interface Files (EIFs)	9	Low x 5	=	45	45
	0	Average x 7	=	0	
	0	High x 10	=	0	
Total Unadjusted Function Point Count					424

Software Project Planning

$$\sum_{i=1}^{14} F_i = 3 + \mathbf{4} + 3 + \mathbf{5} + 3 + 3 + 3 + 3 + 3 + 3 + \mathbf{2} + 3 + \mathbf{0} + 3 = 41$$

$$\begin{aligned} \text{CAF} &= (0.65 + 0.01 \times \sum F_i) \\ &= (0.65 + 0.01 \times 41) \\ &= 1.06 \end{aligned}$$

$$\begin{aligned} \text{FP} &= \text{UFP} \times \text{CAF} \\ &= 424 \times 1.06 \\ &= 449.44 \end{aligned}$$

Hence $\text{FP} = 449$

Software Project Planning

Question: A software project is estimated at 400 FP. A team of five members (one project manager, two senior developers, one junior developer, and one tester) is assigned. Their monthly salaries are ₹90,000, ₹70,000, ₹50,000, and ₹45,000 respectively. The average productivity is 10 FP per person-month. Calculate the total cost of the project.

Solution:

1. Total Person-Months Required

$$\begin{aligned}\text{Effort (PM)} &= \text{Project size (FP)} / \text{Productivity (FP/PM)} \\ &= 400/10 = 40 \text{ person-months}\end{aligned}$$

2. Cost per Person-Month

- Average monthly cost per team member:

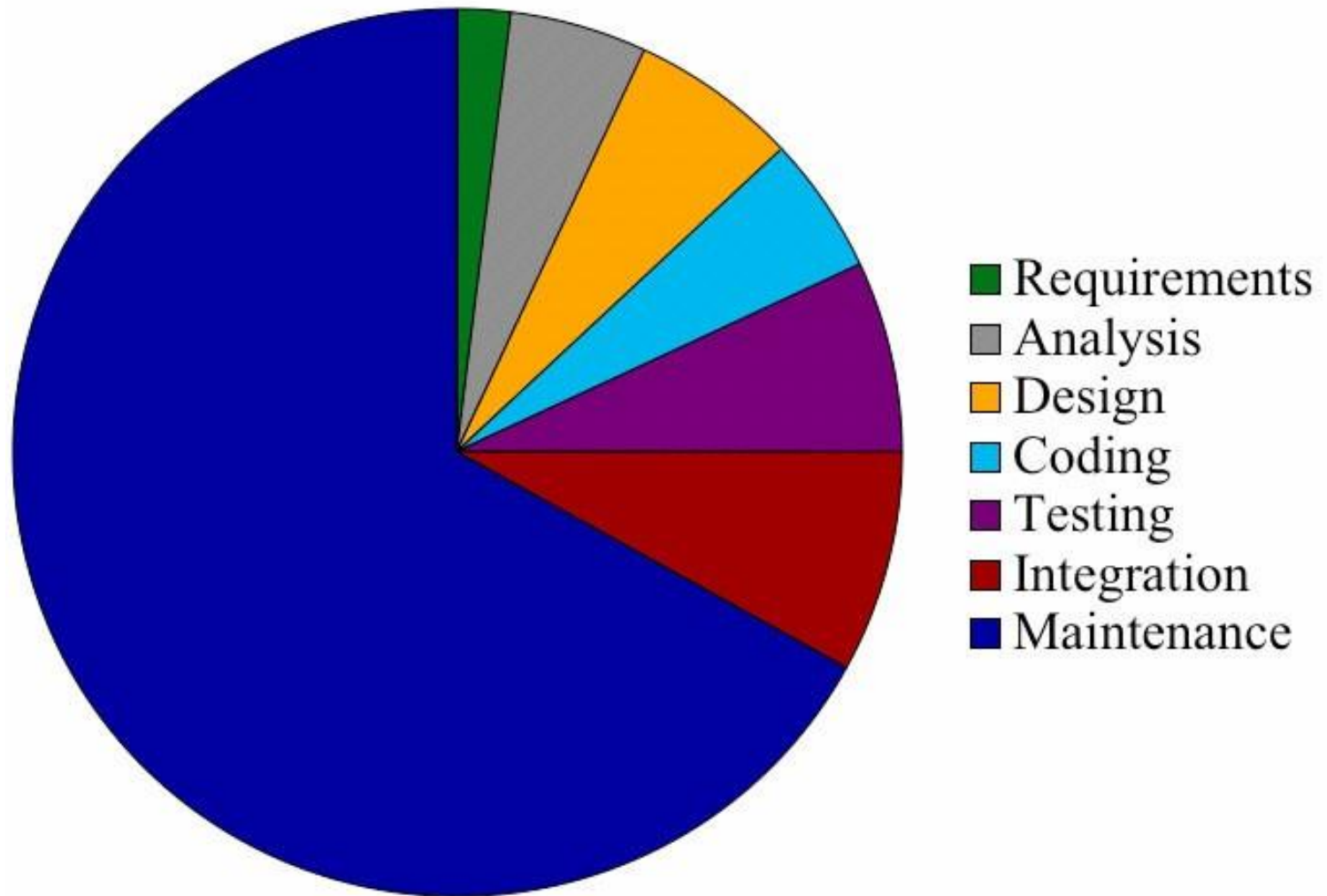
$$\begin{aligned}\text{Average Salary} &= (90,000 + 70,000 + 70,000 + 50,000 + 45,000) / 5 \\ &= 65,000\end{aligned}$$

3. Total Cost

- Total Cost = $40 \times 65,000 = ₹26,00,000$

Software Project Planning

Relative Cost of Software Phases



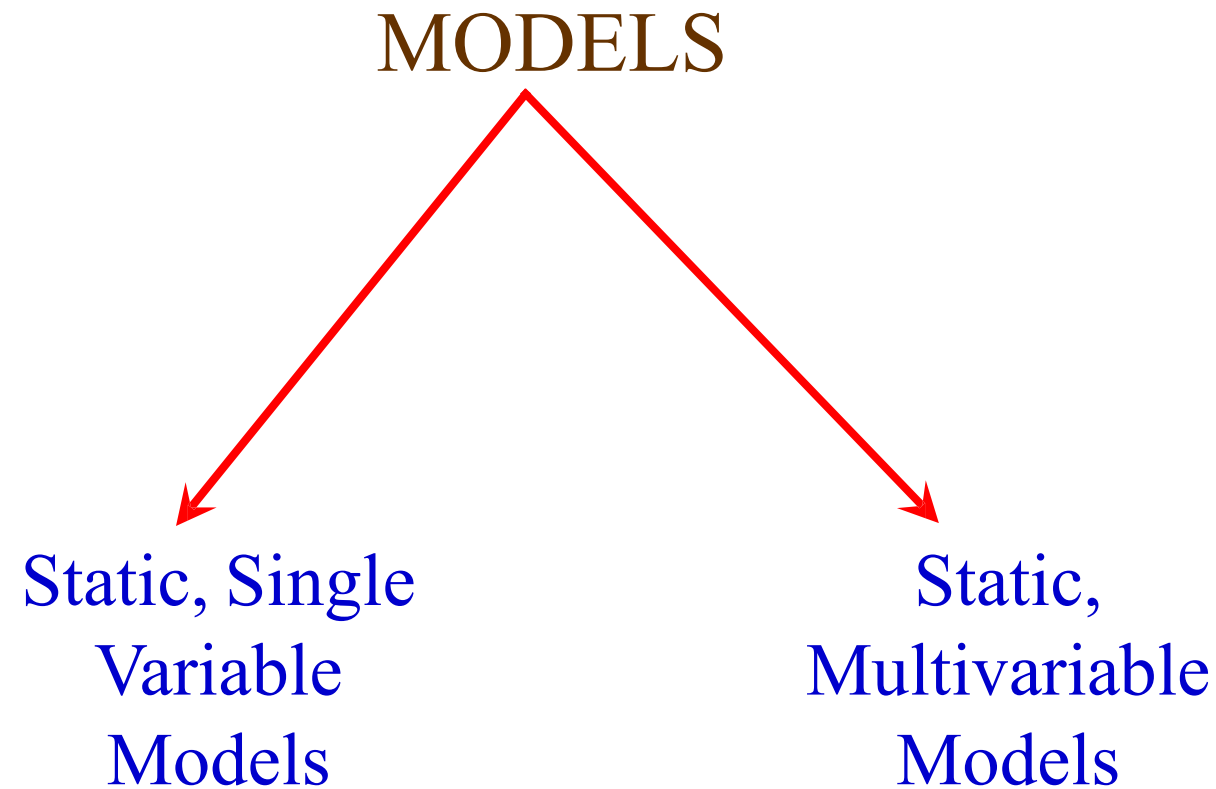
Software Project Planning

Estimation:

Software Estimation is the process of estimating various resources required for completion of a project.

- (i) Estimation of Efforts:
- (ii) Estimation of Schedules:
- (iii) Estimation of Cost:

Software Project Planning



Software Project Planning

Static. Single Variable Models

Methods using this model use an equation to estimate the desired values such as cost, time, effort, etc. They all depend on the same variable used as predictor (say, size). An example of the most common equations is :

$$C = a L^b \quad (i)$$

C is the cost, L is the size and a,b are constants

$$E = 1.4 L^{0.93}$$

$$DOC = 30.4 L^{0.90}$$

$$D = 4.6 L^{0.26}$$

Effort (E in Person-months), documentation (DOC, in number of pages) and duration (D, in months) are calculated from the number of lines of code (L, in thousands of lines) used as a predictor.

Software Project Planning

Static. Multivariable Models

These models are often based on equation (i), they actually depend on several variables representing various aspects of the software development environment, for example method used, user participation, customer oriented changes, memory constraints, etc.

$$E = 5.2 L^{0.91}$$

$$D = 4.1 L^{0.36}$$

Software Project Planning

Example: 4.4

Compare the Walston-Felix model with the SEL model on a software development expected to involve 8 person-years of effort.

- (a) Calculate the number of lines of source code that can be produced.
- (b) Calculate the duration of the development.
- (c) Calculate the productivity in LOC/PY
- (d) Calculate the average manning

Software Project Planning

Solution

The amount of manpower involved = 8 PY = 96 person-months

(a) Number of lines of source code can be obtained by reversing equation to give:

$$L = (E/a)^{1/b}$$

Then

$$L(\text{SEL}) = (96/1.4)^{1/0.93} = 94264 \text{ LOC or}$$

$$94.264 \text{ KLOC.}$$

$$L(\text{W-F}) = (96/5.2)^{1/0.91} = 24632 \text{ LOC or}$$

$$24.632 \text{ KLOC.}$$

Software Project Planning

(b) Duration in months can be calculated by means of equation

$$D(SEL) = 4.6 (L)^{0.26}$$

$$= 4.6 (94.264)^{0.26} = 15 \text{ months}$$

$$D(W-F) = 4.1 L^{0.36}$$

$$= 4.1(24.632)^{0.36} = 13 \text{ months}$$

(c) Productivity is the lines of code produced per person/month (year)

$$P(SEL) = \frac{94264}{8} = 11783 \text{ LOC} / \text{Person} - \text{Years}$$

$$P(W - F) = \frac{24632}{8} = 3079 \text{ LOC} / \text{Person} - \text{Years}$$

Software Project Planning

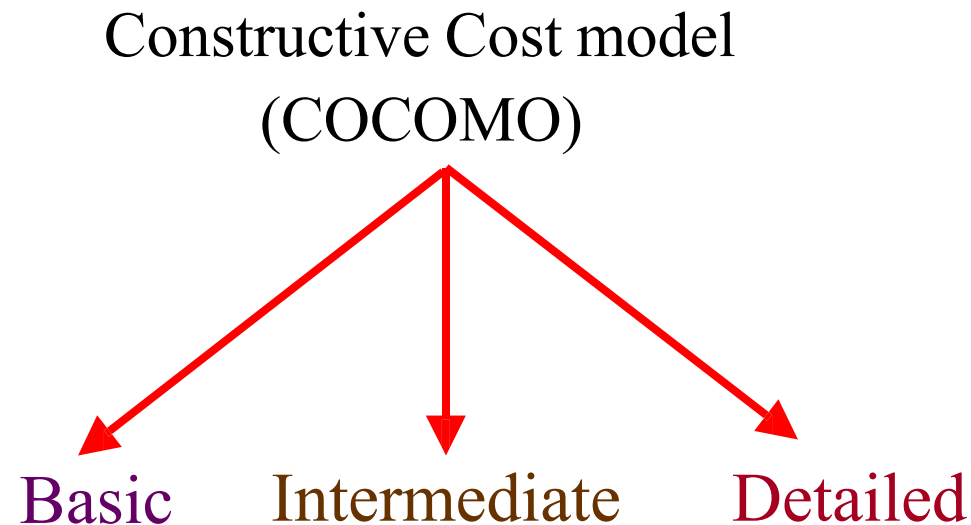
(d) Average manning is the average number of persons required per month in the project.

$$M(SEL) = \frac{96P - M}{15M} = 6.4Persons$$

$$M(W - F) = \frac{96P - M}{13M} = 7.4Persons$$

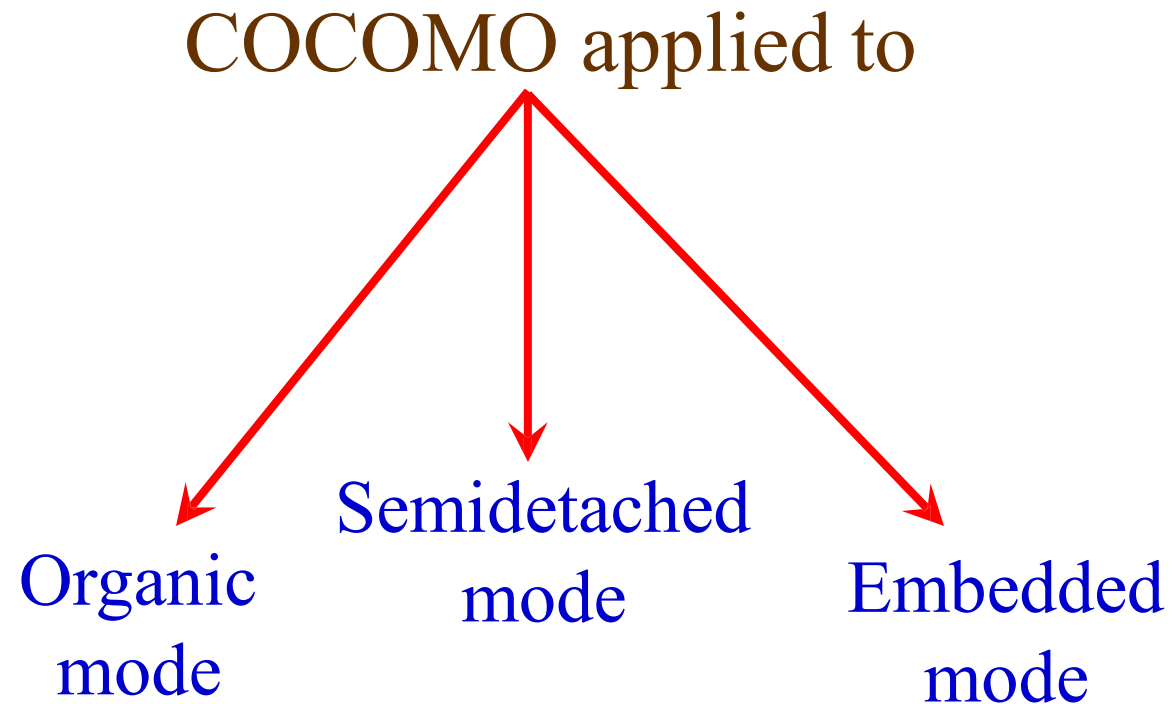
Software Project Planning

The Constructive Cost Model (COCOMO)



Model proposed by
B. W. Boehm's
through his book
Software Engineering Economics in 1981

Software Project Planning



Software Project Planning

Mode	Project size	Nature of Project	Innovation	Deadline of the project	Development Environment
Organic	Typically 2-50 KLOC	Small size project, experienced developers in the familiar environment. For example, pay roll, inventory projects etc.	Little	Not tight	Familiar & In house
Semi detached	Typically 50-300 KLOC	Medium size project, Medium size team, Average previous experience on similar project. For example: Utility systems like compilers, database systems, editors etc.	Medium	Medium	Medium
Embedded	Typically over 300 KLOC	Large project, Real time systems, Complex interfaces, Very little previous experience. For example: ATMs, Air Traffic Control etc.	Significant	Tight	Complex Hardware/ customer Interfaces required

Table 4: The comparison of three COCOMO modes

Software Project Planning

Basic Model

Basic COCOMO model takes the form

$$E = a_b (KLOC)^{b_b}$$

$$D = c_b (E)^{d_b}$$

where E is effort applied in Person-Months, and D is the development time in months. The coefficients a_b , b_b , c_b and d_b are given in table 4 (a).

Software Project Planning

Software Project	a_b	b_b	c_b	d_b
Organic	2.4	1.05	2.5	0.38
Semidetached	3.0	1.12	2.5	0.35
Embedded	3.6	1.20	2.5	0.32

Table 4(a): Basic COCOMO coefficients

Software Project Planning

When effort and development time are known, the average staff size to complete the project may be calculated as:

$$\text{Average staff size (SS)} = \frac{E}{D} \text{ Persons}$$

When project size is known, the productivity level may be calculated as:

$$\text{Productivity (P)} = \frac{KLOC}{E} KLOC / PM$$

Software Project Planning

Example: 4.5

Suppose that a project was estimated to be 400 KLOC. Calculate the effort and development time for each of the three modes i.e., organic, semidetached and embedded.

Software Project Planning

Solution

The basic COCOMO equation take the form:

$$E = a_b (KLOC)^{b_b}$$

$$D = c_b (KLOC)^{d_b}$$

Estimated size of the project = 400 KLOC

(i) Organic mode

$$E = 2.4(400)^{1.05} = 1295.31 \text{ PM}$$

$$D = 2.5(1295.31)^{0.38} = 38.07 \text{ M}$$

Software Project Planning

(ii) Semidetached mode

$$E = 3.0(400)^{1.12} = 2462.79 \text{ PM}$$

$$D = 2.5(2462.79)^{0.35} = 38.45 \text{ M}$$

(iii) Embedded mode

$$E = 3.6(400)^{1.20} = 4772.81 \text{ PM}$$

$$D = 2.5(4772.8)^{0.32} = 38 \text{ M}$$

Software Project Planning

Example: 4.6

A project size of 200 KLOC is to be developed. Software development team has average experience on similar type of projects. The project schedule is not very tight. Calculate the effort, development time, average staff size and productivity of the project.

Software Project Planning

Solution

The semi-detached mode is the most appropriate mode; keeping in view the size, schedule and experience of the development team.

Hence $E = 3.0(200)^{1.12} = 1133.12 \text{ PM}$

$$D = 2.5(1133.12)^{0.35} = 29.3 \text{ M}$$

$$\text{Average staff size } (SS) = \frac{E}{D} \text{ Persons}$$

$$= \frac{1133.12}{29.3} = 38.67 \text{ Persons}$$

Software Project Planning

$$\text{Productivity} = \frac{KLOC}{E} = \frac{200}{1133.12} = 0.1765 KLOC / PM$$

$$P = 176 LOC / PM$$

Software Project Planning

Intermediate Model

Cost drivers

(i) Product Attributes

- Required s/w reliability (RELY)
- Size of application database (DATA)
- Complexity of the product (CPLX)

(ii) Hardware Attributes

- Run time performance constraints (TIME)
- Memory constraints (STOR)
- Virtual machine volatility (VIRT)
- Turnaround time (TURN)

Software Project Planning

(iii) Personal Attributes

- Analyst capability (ACAP)
- Application experience (AEXP)
- Programmer capability (PCAP)
- Virtual m/c experience (VEXP)
- Programming language experience (LEXP)

(iv) Project Attributes

- Modern programming practices (MODP)
- Use of software tools (TOOL)
- Required development Schedule (SCED)

Software Project Planning

Multipliers of different cost drivers

Cost Drivers	RATINGS					
	Very low	Low	Nominal	High	Very high	Extra high
Product Attributes						
RELY	0.75	0.88	1.00	1.15	1.40	--
DATA	--	0.94	1.00	1.08	1.16	--
CPLX	0.70	0.85	1.00	1.15	1.30	1.65
Computer Attributes						
TIME	--	--	1.00	1.11	1.30	1.66
STOR	--	--	1.00	1.06	1.21	1.56
VIRT	--	0.87	1.00	1.15	1.30	--
TURN	--	0.87	1.00	1.07	1.15	--

Software Project Planning

Cost Drivers	RATINGS					
	Very low	Low	Nominal	High	Very high	Extra high
Personnel Attributes						
ACAP	1.46	1.19	1.00	0.86	0.71	--
AEXP	1.29	1.13	1.00	0.91	0.82	--
PCAP	1.42	1.17	1.00	0.86	0.70	--
VEXP	1.21	1.10	1.00	0.90	--	--
LEXP	1.14	1.07	1.00	0.95	--	--
Project Attributes						
MODP	1.24	1.10	1.00	0.91	0.82	--
TOOL	1.24	1.10	1.00	0.91	0.83	--
SCED	1.23	1.08	1.00	1.04	1.10	--

Table 5: Multiplier values for effort calculations

Software Project Planning

Intermediate COCOMO equations

$$E = a_i (KLOC)^{b_i} * EAF$$

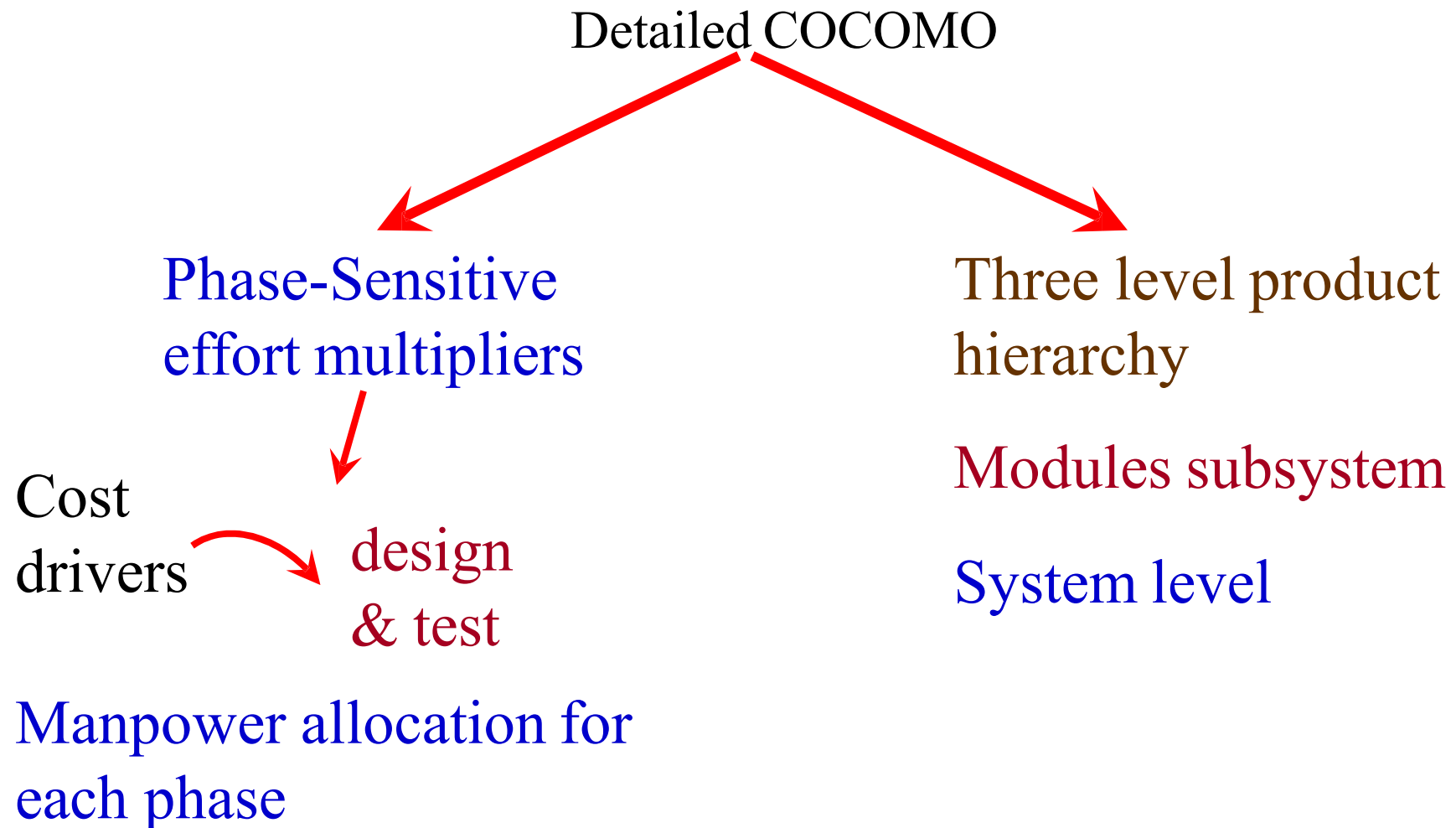
$$D = c_i (E)^{d_i}$$

Project	a_i	b_i	c_i	d_i
Organic	3.2	1.05	2.5	0.38
Semidetached	3.0	1.12	2.5	0.35
Embedded	2.8	1.20	2.5	0.32

Table 6: Coefficients for intermediate COCOMO

Software Project Planning

Detailed COCOMO Model



Software Project Planning

Development Phase

Plan / Requirements

EFFORT : 6% to 8%

DEVELOPMENT TIME : 10% to 40%

% depend on mode & size

Software Project Planning

Design

Effort : 16% to 18%
Time : 19% to 38%

Programming

Effort : 48% to 68%
Time : 24% to 64%

Integration & Test

Effort : 16% to 34%
Time : 18% to 34%

Software Project Planning

Principle of the effort estimate

Size equivalent

As the software might be partly developed from software already existing (that is, re-usable code), a full development is not always required. In such cases, the parts of design document (DD%), code (C%) and integration (I%) to be modified are estimated. Then, an adjustment factor, A, is calculated by means of the following equation.

$$A = 0.4 \text{ DD} + 0.3 \text{ C} + 0.3 \text{ I}$$

The size equivalent is obtained by

$$S \text{ (equivalent)} = (S \times A) / 100$$

$$E_p = \mu_p E$$

$$D_p = \tau_p D$$

Software Project Planning

Lifecycle Phase Values of μ_p

Mode & Code Size	Plan & Requirements	System Design	Detailed Design	Module Code & Test	Integration & Test
Organic Small $S \approx 2$	0.06	0.16	0.26	0.42	0.16
Organic medium $S \approx 32$	0.06	0.16	0.24	0.38	0.22
Semidetached medium $S \approx 32$	0.07	0.17	0.25	0.33	0.25
Semidetached large $S \approx 128$	0.07	0.17	0.24	0.31	0.28
Embedded large $S \approx 128$	0.08	0.18	0.25	0.26	0.31
Embedded extra large $S \approx 320$	0.08	0.18	0.24	0.24	0.34

Table 7 : Effort and schedule fractions occurring in each phase of the lifecycle

Software Project Planning

Lifecycle Phase Values of τ_p

Mode & Code Size	Plan & Requirements	System Design	Detailed Design	Module Code & Test	Integration & Test
Organic Small $S \approx 2$	0.10	0.19	0.24	0.39	0.18
Organic medium $S \approx 32$	0.12	0.19	0.21	0.34	0.26
Semidetached medium $S \approx 32$	0.20	0.26	0.21	0.27	0.26
Semidetached large $S \approx 128$	0.22	0.27	0.19	0.25	0.29
Embedded large $S \approx 128$	0.36	0.36	0.18	0.18	0.28
Embedded extra large $S \approx 320$	0.40	0.38	0.16	0.16	0.30

Table 7 : Effort and schedule fractions occurring in each phase of the lifecycle

Software Project Planning

Distribution of software life cycle:

1. Requirement and product design
 - (a) Plans and requirements
 - (b) System design
2. Detailed Design
 - (a) Detailed design
3. Code & Unit test
 - (a) Module code & test
4. Integrate and Test
 - (a) Integrate & Test

Software Project Planning

Example: 4.7

A new project with estimated 400 KLOC embedded system has to be developed. Project manager has a choice of hiring from two pools of developers: Very highly capable with very little experience in the programming language being used

Or

Developers of low quality but a lot of experience with the programming language. What is the impact of hiring all developers from one or the other pool ?

Note: EAF is wrong, please check

Software Project Planning

Solution

This is the case of embedded mode and model is intermediate COCOMO.

$$\begin{aligned}\text{Hence } E &= a_i (KLOC)^{d_i} \\ &= 2.8 (400)^{1.20} = 3712 \text{ PM}\end{aligned}$$

Case I: Developers are very highly capable with very little experience in the programming being used.

$$\text{EAF} = 0.82 \times 1.14 = 0.9348$$

$$E = 3712 \times .9348 = 3470 \text{ PM}$$

$$D = 2.5 (3470)^{0.32} = 33.9 \text{ M}$$

Software Project Planning

Case II: Developers are of low quality but lot of experience with the programming language being used.

$$\text{EAF} = 1.29 \times 0.95 = 1.22$$

$$\text{E} = 3712 \times 1.22 = 4528 \text{ PM}$$

$$\text{D} = 2.5 (4528)^{0.32} = 36.9 \text{ M}$$

Case II requires more effort and time. Hence, low quality developers with lot of programming language experience could not match with the performance of very highly capable developers with very little experience.

Software Project Planning

Example: 4.8

Consider a project to develop a full screen editor. The major components identified are:

- I. Screen edit
- II. Command Language Interpreter
- III. File Input & Output
- IV. Cursor Movement
- V. Screen Movement

The size of these are estimated to be 4k, 2k, 1k, 2k and 3k delivered source code lines. Use COCOMO to determine

1. Overall cost and schedule estimates (assume values for different cost drivers, with at least four of them being different from 1.0)
2. Cost & Schedule estimates for different phases.

Software Project Planning

Solution

Size of five modules are:

Screen edit	= 4 KLOC
Command language interpreter	= 2 KLOC
File input and output	= 1 KLOC
Cursor movement	= 2 KLOC
Screen movement	= 3 KLOC
Total	= 12 KLOC

Software Project Planning

Let us assume that significant cost drivers are

- i. Required software reliability is high, i.e., 1.15
- ii. Product complexity is high, i.e., 1.15
- iii. Analyst capability is high, i.e., 0.86
- iv. Programming language experience is low, i.e., 1.07
- v. All other drivers are nominal

$$\text{EAF} = 1.15 \times 1.15 \times 0.86 \times 1.07 = 1.2169$$

Software Project Planning

(a) The initial effort estimate for the project is obtained from the following equation

$$\begin{aligned} E &= a_i (\text{KLOC})^{b_i} \times \text{EAF} \\ &= 3.2(12)^{1.05} \times 1.2169 = 52.91 \text{ PM} \end{aligned}$$

Development time

$$\begin{aligned} D &= C_i (E)^{d_i} \\ &= 2.5(52.91)^{0.38} = 11.29 \text{ M} \end{aligned}$$

(b) Using the following equations and referring Table 7, phase wise cost and schedule estimates can be calculated.

$$E_p = \mu_p E$$

$$D_p = \tau_p D$$

Software Project Planning

Since size is only 12 KLOC, it is an organic small model. Phase wise effort distribution is given below:

System Design	$= 0.16 \times 52.91 = 8.465 \text{ PM}$
Detailed Design	$= 0.26 \times 52.91 = 13.756 \text{ PM}$
Module Code & Test	$= 0.42 \times 52.91 = 22.222 \text{ PM}$
Integration & Test	$= 0.16 \times 52.91 = 8.465 \text{ Pm}$

Now Phase wise development time duration is

System Design	$= 0.19 \times 11.29 = 2.145 \text{ M}$
Detailed Design	$= 0.24 \times 11.29 = 2.709 \text{ M}$
Module Code & Test	$= 0.39 \times 11.29 = 4.403 \text{ M}$
Integration & Test	$= 0.18 \times 11.29 = 2.032 \text{ M}$

Software Project Planning

COCOMO-II

The following categories of applications / projects are identified by COCOMO-II and are shown in fig. 4 shown below:

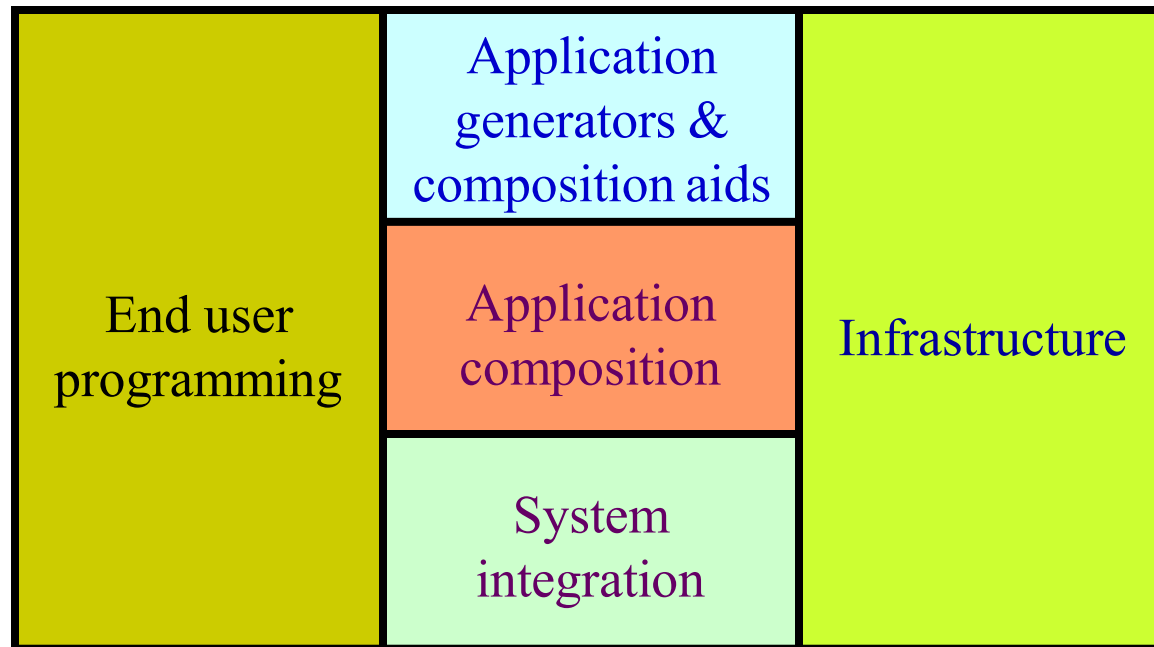


Fig. 4 : Categories of applications / projects

Software Project Planning

Stage No	Model Name	Application for the types of projects	Applications
Stage I	Application composition estimation model	Application composition	In addition to application composition type of projects, this model is also used for prototyping (if any) stage of application generators, infrastructure & system integration.
Stage II	Early design estimation model	Application generators, infrastructure & system integration	Used in early design stage of a project, when less is known about the project.
Stage III	Post architecture estimation model	Application generators, infrastructure & system integration	Used after the completion of the detailed architecture of the project.

Table 8: Stages of COCOMO-II

Software Project Planning

Application Composition Estimation Model

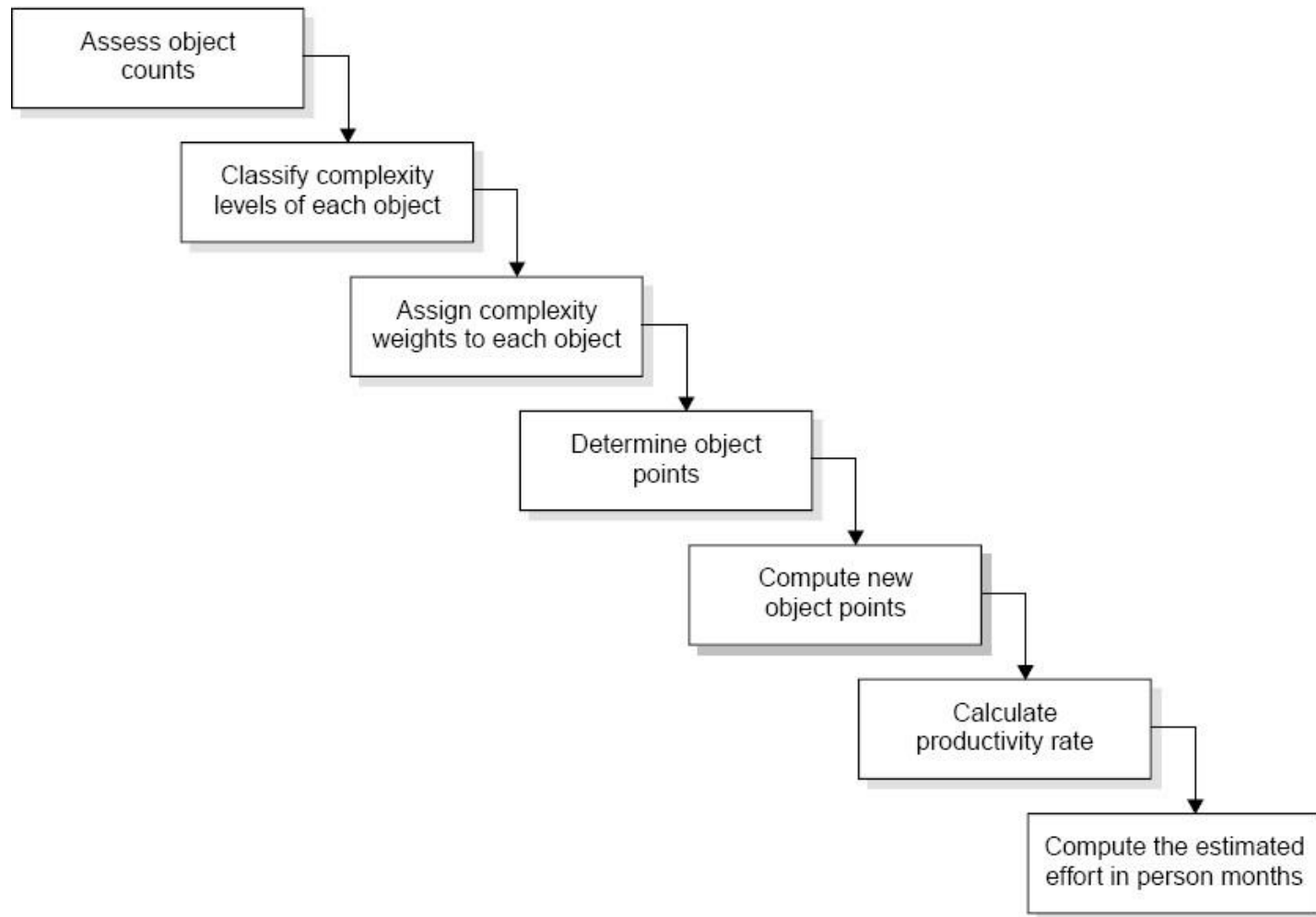


Fig.5: Steps for the estimation of effort in person months

Software Project Planning

- i. **Assess object counts:** Estimate the number of screens, reports and 3 GL components that will comprise this application.
- ii. **Classification of complexity levels:** We have to classify each object instance into simple, medium and difficult complexity levels depending on values of its characteristics.

<i>Number of views contained</i>	<i># and sources of data tables</i>		
	<i>Total < 4 (< 2 server < 3 client)</i>	<i>Total < 8 (2 – 3 server 3 – 5 client)</i>	<i>Total 8 + (> 3 server, > 5 client)</i>
< 3	Simple	Simple	Medium
3 – 7	Simple	Medium	Difficult
> 8	Medium	Difficult	Difficult

Table 9 (a): For screens

Software Project Planning

<i>Number of sections contained</i>	<i># and sources of data tables</i>		
	<i>Total < 4 (< 2 server < 3 client)</i>	<i>Total < 8 (2 – 3 server 3 – 5 client)</i>	<i>Total 8 + (> 3 server, > 5 client)</i>
0 or 1	Simple	Simple	Medium
2 or 3	Simple	Medium	Difficult
4 +	Medium	Difficult	Difficult

Table 9 (b): For reports

Software Project Planning

- iii. Assign complexity weight to each object : The weights are used for three object types i.e., screen, report and 3GL components using the Table 10.

<i>Object Type</i>	<i>Complexity Weight</i>		
	<i>Simple</i>	<i>Medium</i>	<i>Difficult</i>
Screen	1	2	3
Report	2	5	8
3GL Component	—	—	10

Table 10: Complexity weights for each level

Software Project Planning

- iv. **Determine object points:** Add all the weighted object instances to get one number and this known as object-point count.
- v. **Compute new object points:** We have to estimate the percentage of reuse to be achieved in a project. Depending on the percentage reuse, the new object points (NOP) are computed.

$$\text{NOP} = \frac{(\text{object points}) * (100 - \% \text{reuse})}{100}$$

NOP are the object points that will need to be developed and differ from the object point count because there may be reuse.

Software Project Planning

vi. Calculation of productivity rate: The productivity rate can be calculated as:

$$\text{Productivity rate (PROD)} = \text{NOP/Person month}$$

<i>Developer's experience & capability; ICASE maturity & capability</i>	<i>PROD (NOP/PM)</i>
Very low	4
Low	7
Nominal	13
High	25
Very high	50

Table 11: Productivity values

Software Project Planning

- vii. Compute the effort in Persons-Months: When PROD is known, we may estimate effort in Person-Months as:

$$\text{Effort in PM} = \frac{\text{NOP}}{\text{PROD}}$$

Software Project Planning

Example: 4.9

Consider a database application project with the following characteristics:

- I. The application has 4 screens with 4 views each and 7 data tables for 3 servers and 4 clients.
- II. The application may generate two report of 6 sections each from 07 data tables for two server and 3 clients. There is 10% reuse of object points.

The developer's experience and capability in the similar environment is low. The maturity of organization in terms of capability is also low. Calculate the object point count, New object points and effort to develop such a project.

Software Project Planning

Solution

This project comes under the category of application composition estimation model.

Number of screens = 4 with 4 views each

Number of reports = 2 with 6 sections each

From Table 9 we know that each screen will be of medium complexity and each report will be difficult complexity.

Using Table 10 of complexity weights, we may calculate object point count.

$$= 4 \times 2 + 2 \times 8 = 24$$

$$\text{NOP} = \frac{24 * (100 - 10)}{100} = 21.6$$

Software Project Planning

Table 11 gives the low value of productivity (PROD) i.e. 7.

$$\text{Efforts in PM} = \frac{\text{NOP}}{\text{PROD}}$$

$$\text{Efforts} = \frac{21.6}{7} = 3.086 \text{ PM}$$

Software Project Planning

The Early Design Model

The COCOMO-II models use the base equation of the form

$$PM_{\text{nominal}} = A * (\text{size})^B$$

where

PM_{nominal} = Effort of the project in person months

A = Constant representing the nominal productivity, provisionally set to 2.5

B = Scale factor

Size = Software size

Software Project Planning

Scale factor	Explanation	Remarks
Precedentness	Reflects the previous experience on similar projects. This is applicable to individuals & organization both in terms of expertise & experience	Very low means no previous experiences, Extra high means that organization is completely familiar with this application domain.
Development flexibility	Reflect the degree of flexibility in the development process.	Very low means a well defined process is used. Extra high means that the client gives only general goals.
Architecture/ Risk resolution	Reflect the degree of risk analysis carried out.	Very low means very little analysis and Extra high means complete and through risk analysis.

Cont...

Table 12: Scaling factors required for the calculation of the value of B

Software Project Planning

Scale factor	Explanation	Remarks
Team cohesion	Reflects the team management skills.	Very low means no previous experiences, Extra high means that organization is completely familiar with this application domain.
Process maturity	Reflects the process maturity of the organization. Thus it is dependent on SEI-CMM level of the organization.	Very low means organization has no level at all and extra high means organization is related as highest level of SEI-CMM.

Table 12: Scaling factors required for the calculation of the value of B

Software Project Planning

Scaling factors	Very low	Low	Nominal	High	Very high	Extra high
Precedent ness	6.20	4.96	3.72	2.48	1.24	0.00
Development flexibility	5.07	4.05	3.04	2.03	1.01	0.00
Architecture/ Risk resolution	7.07	5.65	4.24	2.83	1.41	0.00
Team cohesion	5.48	4.38	3.29	2.19	1.10	0.00
Process maturity	7.80	6.24	4.68	3.12	1.56	0.00

Table 13: Data for the Computation of B

The value of B can be calculated as:

$$B=0.91 + 0.01 * (\text{Sum of rating on scaling factors for the project})$$

Software Project Planning

Early design cost drivers

There are seven early design cost drivers and are given below:

- i. Product Reliability and Complexity (RCPX)
- ii. Required Reuse (RUSE)
- iii. Platform Difficulty (PDIF)
- iv. Personnel Capability (PERS)
- v. Personnel Experience (PREX)
- vi. Facilities (FCIL)
- vii. Schedule (SCED)

Software Project Planning

Post architecture cost drivers

There are 17 cost drivers in the Post Architecture model. These are rated on a scale of 1 to 6 as given below :

<i>Very Low</i>	<i>Low</i>	<i>Nominal</i>	<i>High</i>	<i>Very High</i>	<i>Extra High</i>
1	2	3	4	5	6

The list of seventeen cost drivers is given below :

- i. Reliability Required (RELY)
- ii. Database Size (DATA)
- iii. Product Complexity (CPLX)
- iv. Required Reusability (RUSE)

Software Project Planning

- v. Documentation (DOCU)
- vi. Execution Time Constraint (TIME)
- vii. Main Storage Constraint (STOR)
- viii. Platform Volatility (PVOL)
- ix. Analyst Capability (ACAP)
- x. Programmers Capability (PCAP)
- xi. Personnel Continuity (PCON)
- xii. Analyst Experience (AEXP)

Software Project Planning

xiii. Programmer Experience (PEXP)

xiv. Language & Tool Experience (LTEX)

xv. Use of Software Tools (TOOL)

xvi. Site Locations & Communication Technology between Sites (SITE)

xvii. Schedule (SCED)

Software Project Planning

Mapping of early design cost drivers and post architecture cost drivers

The 17 Post Architecture Cost Drivers are mapped to 7 Early Design Cost Drivers and are given in Table 14

Early Design Cost Drivers	Counter part Combined Post Architecture Cost drivers
RCPX	RELY, DATA, CPLX, DOCU
RUSE	RUSE
PDIF	TIME, STOR, PVOL
PERS	ACAP, PCAP, PCON
PREX	AEXP, PEXP, LTEX
FCIL	TOOL, SITE
SCED	SCED

Table 14: Mapping table

Software Project Planning

Product of cost drivers for early design model

- i. **Product Reliability and Complexity (RCPX):** The cost driver combines four Post Architecture cost drivers which are RELY, DATA, CPLX and DOCU.

<i>RCPX</i>	<i>Extra Low</i>	<i>Very Low</i>	<i>Low</i>	<i>Nominal</i>	<i>High</i>	<i>Very High</i>	<i>Extra High</i>
Sum of RELY, DATA, CPLX, DOCU ratings	5, 6	7, 8	9-11	12	13-15	16-18	19-21
Emphasis on reliability, documentation	Very Little	Little	Some	Basic	Strong	Very Strong	Extreme
Product complexity	Very Simple	Simple	Some	Moderate	Complex	Very Complex	Extremely Complex
Database size	Small	Small	Small	Moderate	Large	Very Large	Very Large

Software Project Planning

ii. **Required Reuse (RUSE)** : This early design model cost driver is same as its Post architecture Counterpart. The RUSE rating levels are (as per Table 16):

	<i>Vary Low</i>	<i>Low</i>	<i>Nominal</i>	<i>High</i>	<i>Very High</i>	<i>Extra High</i>
	1	2	3	4	5	6
RUSE		None	Across project	Across program	Across product line	Across multiple product line

Software Project Planning

iii. Platform Difficulty (PDIF) : This cost driver combines TIME, STOR and PVOL of Post Architecture Cost Drivers.

<i>PDIF</i>	<i>Low</i>	<i>Nominal</i>	<i>High</i>	<i>Very High</i>	<i>Extra High</i>
Sum of Time, STOR & PVOL ratings	8	9	10-12	13-15	16-17
Time & storage constraint	$\leq 50\%$	$\leq 50\%$	65%	80%	90%
Platform Volatility	Very stable	Stable	Somewhat stable	Volatile	Highly Volatile

Software Project Planning

iv. **Personnel Capability (PERS)** : This cost driver combines three Post Architecture Cost Drivers. These drivers are ACAP, PCAP and PCON.

<i>PERS</i>	<i>Extra Low</i>	<i>Very Low</i>	<i>Low</i>	<i>Nominal</i>	<i>High</i>	<i>Very High</i>	<i>Extra High</i>
Sum of ACAP, PCAP, PCON ratings	3, 4	5, 6	7, 8	9	10, 11	12, 13	14, 15
Combined ACAP & PCAP Percentile	20%	39%	45%	55%	65%	75%	85%
Annual Personnel Turnover	45%	30%	20%	12%	9%	5%	4%

Software Project Planning

- v. **Personnel Experience (PREX)** : This early design driver combines three Post Architecture Cost Drivers, which are AEXP, PEXP and LTEX.

<i>PREX</i>	<i>Extra Low</i>	<i>Very Low</i>	<i>Low</i>	<i>Nominal</i>	<i>High</i>	<i>Very High</i>	<i>Extra High</i>
Sum of AEXP, PEXP and LTEX ratings	3, 4	5, 6	7, 8	9	10, 11	12, 13	14, 15
Applications, Platform, Language & Tool Experience	≤ 3 months	5 months	9 months	1 year	2 year	4 year	6 year

Software Project Planning

vi. **Facilities (FCIL):** This depends on two Post Architecture Cost Drivers, which are TOOL and SITE.

<i>FCIL</i>	<i>Extra Low</i>	<i>Very Low</i>	<i>Low</i>	<i>Nominal</i>	<i>High</i>	<i>Very High</i>	<i>Extra High</i>
Sum of TOOL & SITE ratings	2	3	4, 5	6	7, 8	9, 10	11
Tool support	Minimal	Some	Simple CASE tools	Basic life cycle tools	Good support of tools	Very strong use of tools	Very strong & well integrated tools
Multisite conditions development support	Weak support of complex multisite development	Some support	Moderate support	Basic support	Strong support	Very strong support	Very strong support

Software Project Planning

vii. **Schedule (SCED)** : This early design cost driver is the same as Post Architecture Counterpart and rating level are given below using table 16.

<i>SCED</i>	<i>Very Low</i>	<i>Low</i>	<i>Nominal</i>	<i>High</i>	<i>Very High</i>
Schedule	75% of Nominal	85%	100%	130%	160%

Software Project Planning

The seven early design cost drivers have been converted into numeric values with a Nominal value 1.0. These values are used for the calculation of a factor called “Effort multiplier” which is the product of all seven early design cost drivers. The numeric values are given in Table 15.

<i>Early design Cost drivers</i>	<i>Extra Low</i>	<i>Very Low</i>	<i>Low</i>	<i>Nominal</i>	<i>High</i>	<i>Very High</i>	<i>Extra High</i>
RCPX	.73	.81	.98	1.0	1.30	1.74	2.38
RUSE	—	—	0.95	1.0	1.07	1.15	1.24
PDIF	—	—	0.87	1.0	1.29	1.81	2.61
PERS	2.12	1.62	1.26	1.0	0.83	0.63	0.50
PREX	1.59	1.33	1.12	1.0	0.87	0.71	0.62
FCIL	1.43	1.30	1.10	1.0	0.87	0.73	0.62
SCED	—	1.43	1.14	1.0	1.0	1.0	—

Table 15: Early design parameters

Software Project Planning

The early design model adjusts the nominal effort using 7 effort multipliers (EMs). Each effort multiplier (also called drivers) has 7 possible weights as given in Table 15. These factors are used for the calculation of adjusted effort as given below:

$$PM_{adjusted} = PM_{nominal} \times \prod_{i=1}^7 EM_i$$

$PM_{adjusted}$ effort may vary even up to 400% from $PM_{nominal}$

Hence $PM_{adjusted}$ is the fine tuned value of effort in the early design phase

Software Project Planning

Example: 4.10

A software project of application generator category with estimated 50 KLOC has to be developed. The scale factor (B) has low precedentness, high development flexibility and low team cohesion. Other factors are nominal. The early design cost drivers like platform difficult (PDIF) and Personnel Capability (PERS) are high and others are nominal. Calculate the effort in person months for the development of the project.

Software Project Planning

Solution

$$\begin{aligned}\text{Here } B &= 0.91 + 0.01 * (\text{Sum of rating on scaling factors for the project}) \\ &= 0.91 + 0.01 * (4.96 + 2.03 + 4.24 + 4.38 + 4.68) \\ &= 0.91 + 0.01(20.29)=1.1129\end{aligned}$$

$$\begin{aligned}\text{PM}_{\text{nominal}} &= A * (\text{size})^B \\ &= 2.5 * (50)^{1.1129} = 194.41 \text{ Person months}\end{aligned}$$

The 7 cost drivers are

PDIF = high (1.29)

PERS = high (0.83)

RCPX = nominal (1.0)

RUSE = nominal (1.0)

PREX = nominal (1.0)

FCIL = nominal (1.0)

SCEO = nominal (1.0)

Software Project Planning

$$PM_{adjusted} = PM_{nominal} \times \left[\prod_{i=7}^7 EM_i \right]$$

$$= 194.41 * [1.29 \times 0.83)$$

$$= 194.41 \times 1.07$$

$$= 208.155 \text{ Person months}$$

Software Project Planning

Post Architecture Model

The post architecture model is the most detailed estimation model and is intended to be used when a software life cycle architecture has been completed. This model is used in the development and maintenance of software products in the application generators, system integration or infrastructure sectors.

$$PM_{adjusted} = PM_{nominal} \times \prod_{i=1}^{17} EM_i$$

EM : Effort multiplier which is the product of 17 cost drivers.

The 17 cost drivers of the Post Architecture model are described in the table 16.

Software Project Planning

<i>Cost driver</i>	<i>Purpose</i>	<i>Very low</i>	<i>Low</i>	<i>Nominal</i>	<i>High</i>	<i>Very High</i>	<i>Extra High</i>
RELY (Reliability required)	Measure of the extent to which the software must perform its intended function over a period of time	Only slight inconvenience	Low, easily recoverable losses	Moderate, easily recoverable losses	High financial loss	Risk to human life	—
DATA (Data base size)	Measure the affect of large data requirements on product development	—	$\frac{\text{Database size(D)}}{\text{Prog. size (P)}} < 10$	$10 \leq \frac{D}{P} < 100$	$100 \leq \frac{D}{P} < 1000$	$\frac{D}{P} \geq 1000$	—
CPLX (Product complexity)	Complexity is divided into five areas: Control operations, computational operations, device dependent operations, data management operations & User Interface management operations.	See Table 4.17					
DOCU Documentation	Suitability of the project's documentation to its life cycle needs	Many life cycle needs uncovered	Some needs uncovered	Adequate	Excessive for life cycle needs	Very Excessive	—

Table 16: Post Architecture Cost Driver rating level summary

Cont...
104

Software Project Planning

TIME (Execution Time constraint)	Measure of execution time constraint on software	—	—	≤ 50% use of a available execution time	70%	85%	95%
STOR (Main storage constraint)	Measure of main storage constraint on software	—	—	≤ 50% use of available storage	70%	85%	95%
PVOL (Platform Volatility)	Measure of changes to the OS, compilers, editors, DBMS etc.	—	Major changes every 12 months & minor changes every 1 month	Major: 6 months Minor: 2 weeks	Major: 2 months Minor: 1 week	Major: 2 week Minor: 2 days	—
ACAP (Analyst capability)	Should include analysis and design ability, efficiency & thoroughness, and communication skills.	15th Percentile	35th Percentile	55th Percentile	75th Percentile	90th Percentile	—

Table 16: Post Architecture Cost Driver rating level summary

Software Project Planning

PCAP (Programmers capability)	Capability of Programmers as a team. It includes ability, efficiency, thoroughness & communication skills	15th Percentile	35th Percentile	55th Percentile	75th Percentile	90th Percentile	—
PCON (Personnel Continuity)	Rating is in terms of Project's annual personnel turnover	48%/year	24%/year	12%/year	6%/year	3%/year	—
AEXP (Applications Experience)	Rating is dependent on level of applications experience.	≤ 2 months	6 months	1 year	3 year	6 year	—
PEXP (Platform experience)	Measure of Platform experience	≤ 2 months	6 months	1 year	3 year	6 year	—

Table 16: Post Architecture Cost Driver rating level summary

Cont...
106

Software Project Planning

LTEX (Language & Tool experience)	Rating is for Language & tool experience	≤ 2 months	6 months	1 year	3 year	6 year	—
TOOL (Use of software tools)	It is the indicator of usage of software tools	No use	Beginning to use	Some use	Good use	Routine & habitual use	—
SITE (Multisite development)	Site location & Communication technology between sites	International with some phone & mail facility	Multicity & multi company with individual phones, FAX	Multicity & multi company with Narrow band mail	Same city or Metro with wideband electronic communication	Same building or complex with wideband electronic communication & Video conferencing	Fully co-located with interactive multimedia
SCED (Required Development Schedule)	Measure of Schedule constraint. Ratings are defined in terms of percentage of schedule stretch-out or acceleration with respect to nominal schedule	75% of nominal	85%	100%	130%	160%	—

Table 16: Post Architecture Cost Driver rating level summary

Software Project Planning

Product complexity is based on control operations, computational operations, device dependent operations, data management operations and user interface management operations. Module complexity rating are given in table 17.

The numeric values of these 17 cost drivers are given in table 18 for the calculation of the product of efforts i.e., effort multiplier (EM). Hence PM adjusted is calculated which will be a better and fine tuned value of effort in person months.

Software Project Planning

	Control Operations	Computational Operations	Device-dependent Operations	Data management Operations	User Interface Management Operations
Very Low	Straight-line code with a few non-nested structured programming operators: Dos. Simple module composition via procedure calls or simple scripts.	Evaluation of simple expressions: e.g., $A=B+C*(D-E)$	Simple read, write statements with simple formats.	Simple arrays in main memory. Simple COTSDB queries, updates.	Simple input forms, report generators.
Low	Straight forward nesting of structured programming operators. Mostly simple predicates	Evaluation of moderate-level expressions: e.g., $D=\text{SQRT}(B^{**}2-4*A*C)$	No cognizance needed of particular processor or I/O device characteristics. I/O done at GET/PUT level.	Single file sub setting with no data structure changes, no edits, no intermediate files, Moderately complex COTS-DB queries, updates.	User of simple graphics user interface (GUI) builders.

Table 17: Module complexity ratings

Cont...

Software Project Planning

	Control Operations	Computational Operations	Device-dependent Operations	Data management Operations	User Interface Management Operations
Nominal	Mostly simple nesting. Some inter module control Decision tables. Simple callbacks or message passing, including middleware supported distributed processing.	Use of standard maths and statistical routines. Basic matrix/ vector operations.	I/O processing includes device selection, status checking and error processing.	Multi-file input and single file output. Simple structural changes, simple edits. Complex COTS-DB queries, updates.	Simple use of widget set.
High	Highly nested structured programming operators with many compound predicates. Queue and stack control. Homogeneous, distributed processing. Single processor soft real time control.	Basic numerical analysis: multivariate interpolation, ordinary differential equations. Basic truncation, round off concerns.	Operations at physical I/O level (physical storage address translations; seeks, read etc.) Optimized I/O overlap.	Simple triggers activated by data stream contents. Complex data restructuring.	Widget set development and extension. Simple voice I/O multimedia.

Table 17: Module complexity ratings

Cont...

Software Project Planning

	Control Operations	Computational Operations	Device-dependent Operations	Data management Operations	User Interface Management Operations
Very High	Reentrant and recursive coding. Fixed-priority interrupt handling. Task synchronization, complex callbacks, heterogeneous distributed processing. Single processor hard real time control.	Difficult but structured numerical analysis: near singular matrix equations, partial differential equations. Simple parallelization.	Routines for interrupt diagnosis, servicing, masking. Communication line handling. Performance intensive embedded systems.	Distributed database coordination. Complex triggers. Search optimization.	Moderately complex 2D/3D, dynamic graphics, multimedia.
Extra High	Multiple resource scheduling with dynamically changing priorities. Microcode-level control. Distributed hard real time control.	Difficult and unstructured numerical analysis: highly accurate analysis of noisy, stochastic data. Complex parallelization.	Device timing dependent coding, micro programmed operations. Performance critical embedded systems.	Highly coupled, dynamic relational and object structures. Natural language data management.	Complex multimedia, virtual reality.

Table 17: Module complexity ratings

Software Project Planning

Cost Driver	Rating					
	Very Low	Low	Nominal	High	Very High	Extra High
RELY	0.75	0.88	1.00	1.15	1.39	
DATA		0.93	1.00	1.09	1.19	
CPLX	0.75	0.88	1.00	1.15	1.30	1.66
RUSE		0.91	1.00	1.14	1.29	1.49
DOCU	0.89	0.95	1.00	1.06	1.13	
TIME			1.00	1.11	1.31	1.67
STOR			1.00	1.06	1.21	1.57
PVOL		0.87	1.00	1.15	1.30	
ACAP	1.50	1.22	1.00	0.83	0.67	
PCAP	1.37	1.16	1.00	0.87	0.74	

Table 18: 17 Cost Drivers

Cont...

Software Project Planning

Cost Driver	Rating					
	Very Low	Low	Nominal	High	Very High	Extra High
PCON	1.24	1.10	1.00	0.92	0.84	
AEXP	1.22	1.10	1.00	0.89	0.81	
PEXP	1.25	1.12	1.00	0.88	0.81	
LTEX	1.22	1.10	1.00	0.91	0.84	
TOOL	1.24	1.12	1.00	0.86	0.72	
SITE	1.25	1.10	1.00	0.92	0.84	0.78
SCED	1.29	1.10	1.00	1.00	1.00	

Table 18: 17 Cost Drivers

Software Project Planning

Schedule estimation

Development time can be calculated using $PM_{adjusted}$ as a key factor and the desired equation is:

$$TDEV_{nominal} = [\phi \times (PM_{adjusted})^{(0.28+0.2(B-0.091))}] * \frac{SCED\%}{100}$$

where Φ = constant, provisionally set to 3.67

$TDEV_{nominal}$ = calendar time in months with a scheduled constraint

B = Scaling factor

$PM_{adjusted}$ = Estimated effort in Person months (after adjustment)

Software Project Planning

Size measurement

Size can be measured in any unit and the model can be calibrated accordingly. However, COCOMO II details are:

- i. Application composition model uses the size in object points.
- ii. The other two models use size in KLOC

Early design model uses unadjusted function points. These function points are converted into KLOC using Table 19. Post architecture model may compute KLOC after defining LOC counting rules. If function points are used, then use unadjusted function points and convert it into KLOC using Table 19.

Software Project Planning

Language	SLOC/UFP
Ada	71
AI Shell	49
APL	32
Assembly	320
Assembly (Macro)	213
ANSI/Quick/Turbo Basic	64
Basic-Compiled	91
Basic-Interpreted	128
C	128
C++	29

Table 19: Converting function points to lines of code

Software Project Planning

Language	SLOC/UFP
ANSI Cobol 85	91
Fortan 77	105
Forth	64
Jovial	105
Lisp	64
Modula 2	80
Pascal	91
Prolog	64
Report Generator	80
Spreadsheet	6

Table 19: Converting function points to lines of code

Software Project Planning

Example: 4.11

Consider the software project given in example 4.10. Size and scale factor (B) are the same. The identified 17 Cost drivers are high reliability (RELY), very high database size (DATA), high execution time constraint (TIME), very high analyst capability (ACAP), high programmers capability (PCAP). The other cost drivers are nominal. Calculate the effort in Person-Months for the development of the project.

Software Project Planning

Solution

Here $B = 1.1129$

$PM_{\text{nominal}} = 194.41$ Person-months

$$PM_{\text{adjusted}} = PM_{\text{nominal}} \times \prod_{i=7}^{17} EM_i$$

$$= 194.41 \times (1.15 \times 1.19 \times 1.11 \times 0.67 \times 0.87)$$

$$= 194.41 \times 0.885$$

$$= 172.05 \text{ Person-months}$$

Software Project Planning

Putnam Resource Allocation Model

Norden of IBM

Rayleigh curve

Model for a range of hardware development projects.

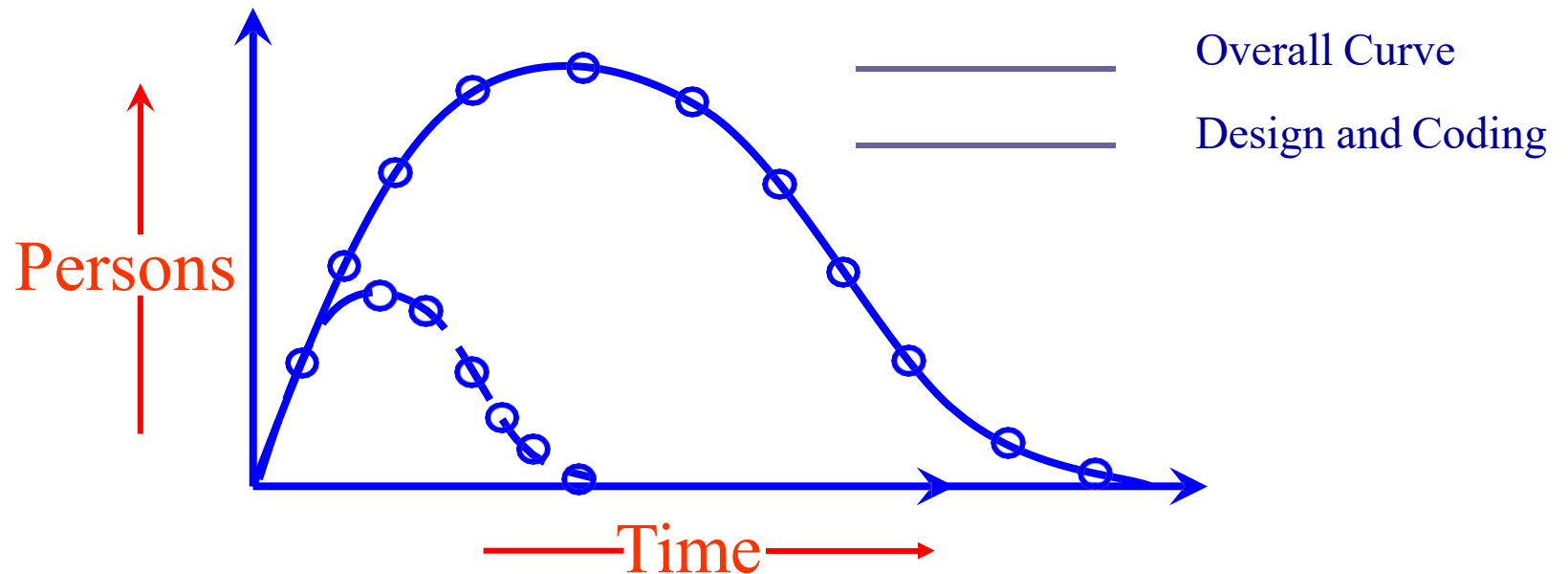


Fig.6: The Rayleigh manpower loading curve

Software Project Planning

Putnam observed that this curve was a close approximation at project level and software subsystem level.

No. of projects = 150

Software Project Planning

The Norden / Rayleigh Curve

The curve is modeled by differential equation

$$m(t) = \frac{dy}{dt} = 2kate^{-at^2} \quad \text{-----} (1)$$

$\frac{dy}{dt}$ = manpower utilization rate per unit time

a = parameter that affects the shape of the curve

K = area under curve in the interval $[0, \infty]$

t = elapsed time

Software Project Planning

On Integration on interval $[0, t]$

$$y(t) = K [1 - e^{-at^2}] \text{ -----(2)}$$

Where $y(t)$: cumulative manpower used upto time t .

$$y(0) = 0$$

$$y(\infty) = k$$

The cumulative manpower is null at the start of the project, and grows monotonically towards the total effort K (area under the curve).

Software Project Planning

$$\frac{d^2 y}{dt^2} = 2kae^{-at^2} [1 - 2at^2] = 0$$

$$t_d^2 = \frac{1}{2a}$$

“ t_d ”: time where maximum effort rate occurs

Replace “ t_d ” for t in equation (2)

$$E = y(t) = k \left[1 - e^{-\frac{t_d^2}{2t_d^2}} \right] = K (1 - e^{-0.5})$$

$$E = y(t) = 0.3935 k$$

$$a = \frac{1}{2t_d^2}$$

Software Project Planning

Replace “a” with $\frac{1}{2t_d^2}$ in the Norden/Rayleigh model. By making this substitution in equation we have

$$\begin{aligned} m(t) &= \frac{2K}{2t_d^2} te^{-\frac{t^2}{2t_d^2}} \\ &= \frac{K}{t_d^2} te^{-\frac{t^2}{2t_d^2}} \end{aligned}$$

Software Project Planning

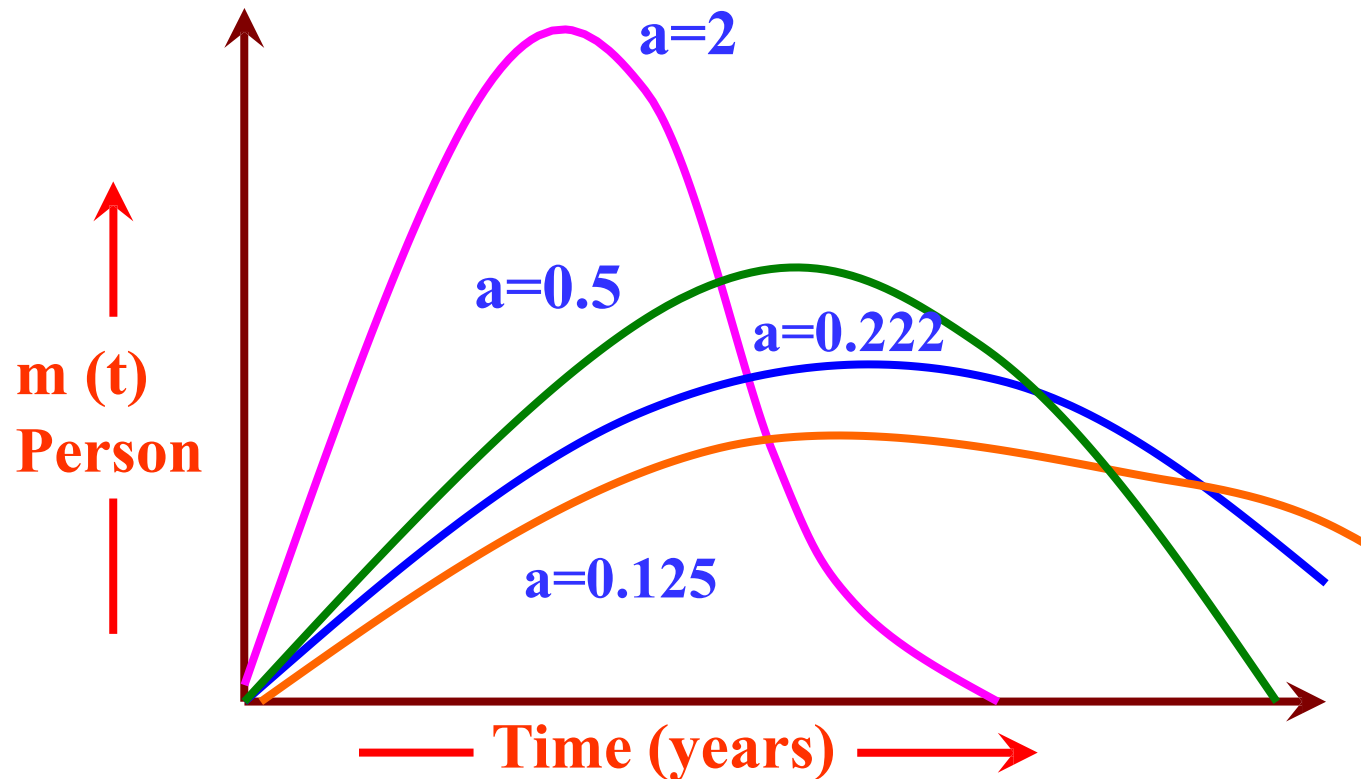


Fig.7: Influence of parameter 'a' on the manpower distribution

Software Project Planning

At time $t=t_d$, peak manning $m(t_d)$ is obtained and denoted by m_o .

$$m_o = \frac{k}{t_d \sqrt{e}}$$

k = Total project cost/effort in person-years.

t_d = Delivery time in years

m_o = No. of persons employed at the peak

e = 2.71828

Software Project Planning

Example: 4.12

A software development project is planned to cost 95 MY in a period of 1 year and 9 months. Calculate the peak manning and average rate of software team build up.

Software Project Planning

Solution

Software development cost

$k=95$ MY

Peak development time

$t_d = 1.75$ years

Peak manning

$$m_o = \frac{k}{t_d \sqrt{e}}$$

$$\frac{95}{1.75 \times 1.648} = 32.94 = 33 \text{ persons}$$

Average rate of software team build up

$$= \frac{m_o}{t_d} = \frac{33}{1.75} = 18.8 \text{ persons / year or } 1.56 \text{ person / month}$$

Software Project Planning

Example: 4.13

Consider a large-scale project for which the manpower requirement is $K=600$ PY and the development time is 3 years 6 months.

- (a) Calculate the peak manning and peak time.
- (b) What is the manpower cost after 1 year and 2 months?

Software Project Planning

Solution

(a) We know $t_d = 3$ years and 6 months = 3.5 years

$$\text{NOW} \quad m_0 = \frac{K}{\sqrt{t_d} e}$$

$$\therefore m_0 = 600 / (3.5 \times 1.648) \cong 104 \text{ persons}$$

Software Project Planning

(b) We know

$$y(t) = K \left[1 - e^{-at^2} \right]$$

$t = 1 \text{ year and } 2 \text{ months}$

$= 1.17 \text{ years}$

$$a = \frac{1}{2t_d^2} = \frac{1}{2 \times (3.5)^2} = 0.041$$

$$y(1.17) = 600 \left[1 - e^{-0.041 (1.17)^2} \right]$$

$= 32.6 \text{ PY}$

Software Project Planning

Difficulty Metric

Slope of manpower distribution curve at start time $t=0$ has some useful properties.

$$m'(t) = \frac{d^2 y}{dt^2} = 2kae^{-at^2} (1 - 2at^2)$$

Then, for $t=0$

$$m'(0) = 2Ka = \frac{2K}{2t_d^2} = \frac{K}{t_d^2}$$

Software Project Planning

The ratio $\frac{K}{t_d^2}$ is called difficulty and denoted by D, which is measured in person/year :

$$D = \frac{k}{t_d^2} \text{ persons/year}$$

Software Project Planning

Project is difficult to develop
if

Manpower demand
is high

When time schedule
is short

Software Project Planning

Peak manning is defined as:

$$m_0 = \frac{k}{t_d \sqrt{e}}$$

$$D = \frac{k}{t_d^2} = \frac{m_0 \sqrt{e}}{t_d}$$

Thus difficult projects tend to have a higher peak manning for a given development time, which is in line with Norden's observations relative to the parameter “a”.

Software Project Planning

Manpower buildup

D is dependent upon “K”. The derivative of D relative to “K” and “ t_d ” are

$$D'(t_d) = \frac{-2k}{t_d^3} \text{ persons / year}^2$$

$$D'(k) = \frac{1}{t_d^2} \text{ year}^{-2}$$

Software Project Planning

$D^1(K)$ will always be very much smaller than the absolute value of $D^1(t_d)$. This difference in sensitivity is shown by considering two projects

Project A : Cost = 20 PY & $t_d = 1$ year

Project B : Cost = 120 PY & $t_d = 2.5$ years

The derivative values are

Project A : $D'(t_d) = -40$ & $D'(K) = 1$

Project B : $D'(t_d) = -15.36$ & $D'(K) = 0.16$

This shows that a given software development is time sensitive.

Software Project Planning

Putnam observed that

Difficulty derivative relative to time



Behavior of s/w development

If project scale is increased, the development time also increase to such an extent that $\frac{k}{t_d^3}$ remains constant

around a value which could be 8,15,27.

Software Project Planning

It is represented by D_0 and can be expressed as:

$$D_0 = \frac{k}{t_d^3} \text{ person / year}^2$$

$D_0 = 8$, new s/w with many interfaces & interactions with other systems.

$D_0 = 15$, New standalone system.

$D_0 = 27$, The software is rebuild form existing software.

Software Project Planning

Example: 4.14

Consider the example 4.13 and calculate the difficulty and manpower build up.

Software Project Planning

Solution

We know

Difficulty $D = \frac{K}{t_d^2}$

$$= \frac{600}{(3.5)^2} = 49 \text{ person / year}$$

Manpower build up can be calculated by following equation

$$D_0 = \frac{K}{t_d^3}$$
$$= \frac{600}{(3.5)^3} = 14 \text{ person / year}^2$$

Software Project Planning

Productivity Versus Difficulty

Productivity = No. of LOC developed per person-month

$$P \propto D^{\beta}$$

Avg. productivity

$$P = \frac{\text{LOC produced}}{\text{cumulative manpower used to produce code}}$$

Software Project Planning

$$P = S/E$$

$$P = \phi D^{-2/3}$$

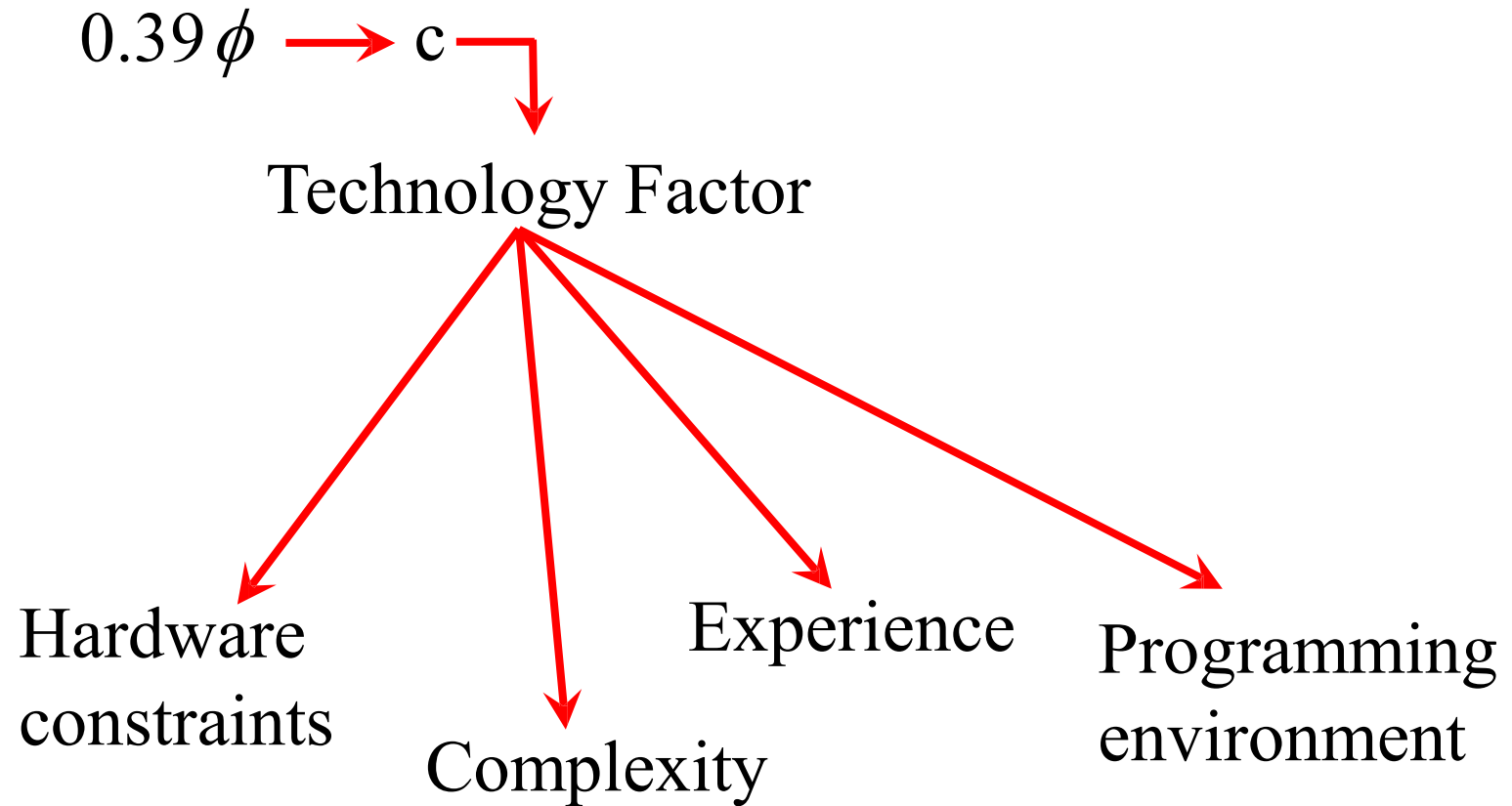
$$S = \phi D^{-2/3} E$$

$$= \phi D^{-2/3} (0.3935 K)$$

$$S = \phi \left[\frac{k}{t_d^2} \right]^{-\frac{2}{3}} k(0.3935)$$

$$S = 0.3935 \phi K^{1/3} t_d^{-4/3}$$

Software Project Planning



Software Project Planning

C $\longrightarrow$ 610 – 57314

K : P-Y

T : Years

$$S = CK^{1/3}t_d^{4/3}$$

$$C = S.K^{-1/3}t_d^{-4/3}$$

The trade off of time versus cost

$$K^{1/3}t_d^{4/3} = S / C$$

$$K = \frac{1}{t_d^4} \left[\frac{S}{C} \right]^3$$

Software Project Planning

$$C = 5000$$

$$S = 5,00,000 \text{ LOC}$$

$$K = \frac{1}{t_d^4} (100)^3$$

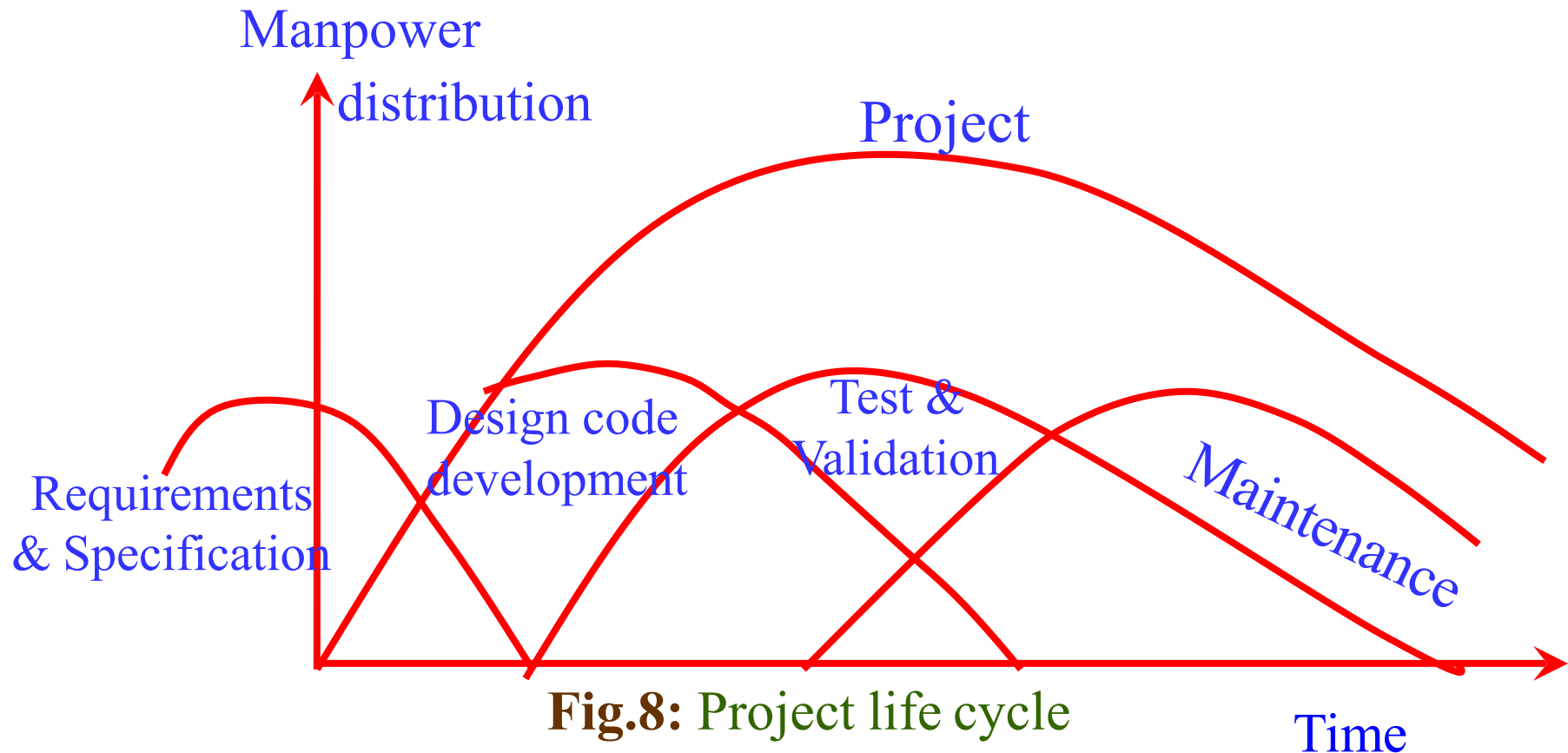
t_d (years)	K (P-Y)
5.0	1600
4.0	3906
3.5	6664
3.0	12346

Table 20: (Manpower versus development time)

Software Project Planning

Development Subcycle

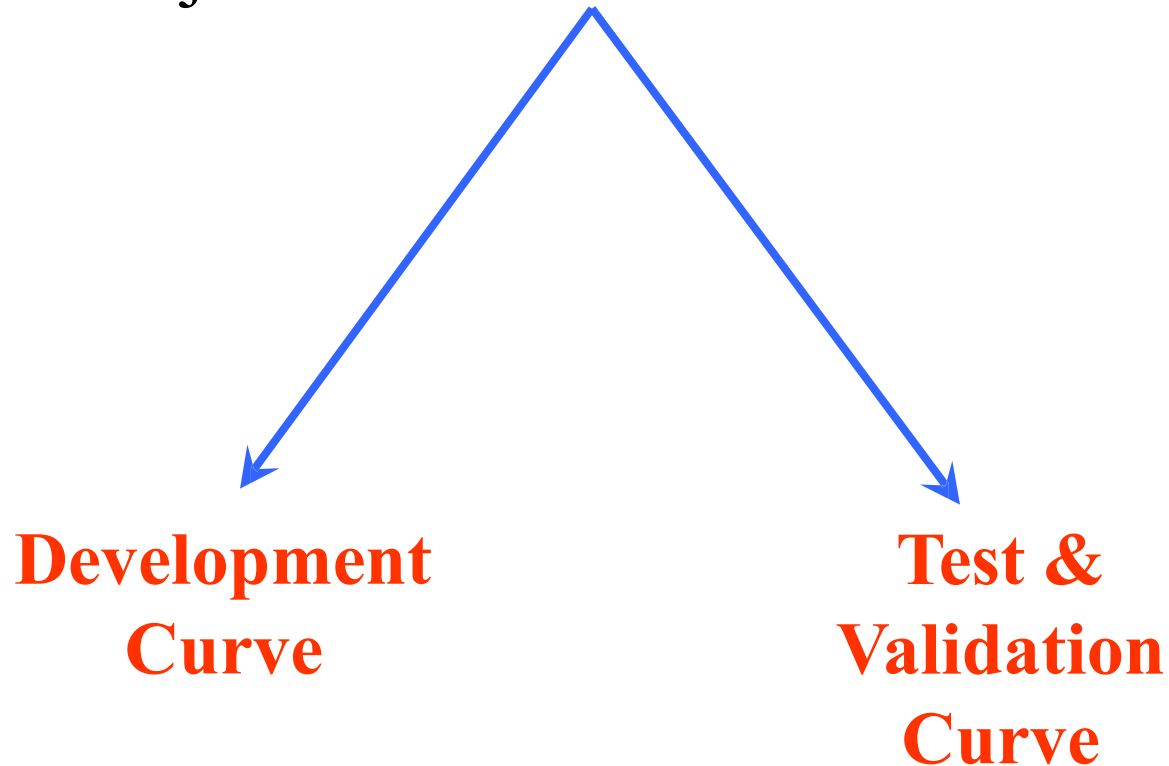
All that has been discussed so far is related to project life cycle as represented by project curve



Software Project Planning

Project life cycle

Project curve is the addition of two curves



Software Project Planning

$$\therefore m_d(t) = 2k_d b t e^{-bt^2}$$
$$y_d(t) = K_d [1 - e^{-bt^2}]$$

An examination of $m_d(t)$ function shows a non-zero value of m_d at time t_d .

This is because the manpower involved in design & coding is still completing this activity after t_d in form of rework due to the validation of the product.

Nevertheless, for the model, a level of completion has to be assumed for development.

It is assumed that 95% of the development will be completed by the time t_d .

Software Project Planning

$$\frac{y_d(t)}{K_d} = 1 - e^{-bt^2} = 0.95$$

$\therefore$ We may say that $b = \frac{1}{2t_{od}^2}$

T_{od} : time at which development curve exhibits a peak manning.

$$t_{od} = \frac{t_d}{\sqrt{6}}$$

Software Project Planning

Relationship between K_d & K must be established.

At the time of origin, both cycles have the same slope.

$$\left[\frac{dm}{dt} \right]_0 = \frac{K}{t_d^2} = \frac{K_d}{t_{od}^2} = \left[\frac{dm_d}{dt} \right]_0$$

$$K_d = K/6$$

$$D = \frac{K}{t_d^2} = \frac{K_d}{t_{od}^2}$$

Software Project Planning

This does not apply to the manpower build up D_0 .

$$D_o = \frac{K}{t_d^3} = \frac{K_d}{\sqrt{6}t_{od}^3}$$

Conte investigated that

Larger projects  reasonable

Medium & small projects  overestimate

Software Project Planning

Example: 4.15

A software development requires 90 PY during the total development sub-cycle. The development time is planned for a duration of 3 years and 5 months

- (a) Calculate the manpower cost expended until development time
- (b) Determine the development peak time
- (c) Calculate the difficulty and manpower build up.

Software Project Planning

Solution

(a) Duration $t_d = 3.41$ years

We know from equation $\frac{y_d(t)}{K_d} = 1 - e^{-bt_d} = 0.95$

$$\frac{y_d(t_d)}{K_d} = 0.95$$

$$Y_d(t_d) = 0.95 \times 90$$

$$= 85.5 \text{ PY}$$

Software Project Planning

(b) We know from equation $t_{od} = \frac{t_d}{\sqrt{6}}$

$$t_{od} = \frac{t_d}{\sqrt{6}} = 3.41 / 2.449 = 1.39 \text{ years}$$

$$\cong 17 \text{ months}$$

Software Project Planning

(c) Total Manpower development

$$K_d = y_d(t_d) / 0.95$$
$$= 85.5 / 0.95 = 90$$

$$K = 6K_d = 90 \times 6 = 540PY$$

$$D = K / t_d^2 = 540 / (3.41)^2 = 46 \text{ persons/years}$$

$$D_o = \frac{K}{t_d^3} = 540 / (3.41)^3 = 13.6 \text{ persons/years}^2$$

Software Project Planning

Example:4.16

A software development for avionics has consumed 32 PY up to development cycle and produced a size of 48000 LOC. The development of project was completed in 25 months. Calculate the development time, total manpower requirement, development peak time, difficulty, manpower build up and technology factor.

Software Project Planning

Solution:

Development time $t_d = 25$ months = 2.08 years

Total manpower development $k_d = \frac{Y_d(t_d)}{0.95} = \frac{32}{0.95} = 33.7 PY$

Development peak time $t_{od} = \frac{(t_d)}{\sqrt{6}} = 0.85 \text{ years} = 10 \text{ months}$

$$K = 6K_d = 6 \times 33.7 = 202 PY$$

$$D = \frac{k}{t_d^2} = \frac{202}{(2.08)^2} = 46.7 \text{ pesons / years}$$

Software Project Planning

$$D_0 = \frac{k}{t_d^3} = \frac{202}{(2.08)^3} = 22.5 \text{ Persons / year}^2$$

Technology factor

$$C = SK^{-1/3} t_d^{-4/3}$$
$$= 3077$$

Software Project Planning

Example 4.17

What amount of software can be delivered in 1 year 10 months in an organization whose technology factor is 2400 if a total of 25 PY is permitted for development effort.

Software Project Planning

Solution:

$$t_d = 1.8 \text{ years}$$

$$K_d = 25 \text{ PY}$$

$$K = 25 \times 6 = 150 \text{ PY}$$

$$C = 2400$$

We know

$$S = CK^{1/3} t_d^{4/3}$$
$$= 2400 \times 5.313 \times 2.18 = 27920 \text{ LOC}$$

Software Project Planning

Example 4.18

The software development organization developing real time software has been assessed at technology factor of 2200. The maximum value of manpower build up for this type of software is $D_o=7.5$. The estimated size to be developed is $S=55000$ LOC.

- (a) Determine the total development time, the total development manpower cost, the difficulty and the development peak manning.
- (b) The development time determined in (a) is considered too long. It is recommended that it be reduced by two months. What would happen?

Software Project Planning

Solution

We have $S = CK^{1/3}t_d^{4/3}$

$$\frac{S^3}{C} = kt_d^4$$

which is also equivalent to $\frac{S^3}{C} = D_o t_d^7$

then $t_d = \left(\frac{1}{D_o C} \frac{S^3}{C} \right)^{1/7}$

Software Project Planning

Since $\frac{S}{C} = 25$

$t_d = 3 \text{ years}$

$$K = D_0 t_d^3 = 7.5 \times 27 = 202 \text{ PY}$$

Total development manpower cost $K_d = \frac{202}{06} = 33.75 \text{ PY}$

$$D = D_0 t_d = 22.5 \text{ persons / year}$$

$$t_{od} = \frac{t_d}{\sqrt{6}} = \frac{3}{\sqrt{6}} = 1.2 \text{ years}$$

Software Project Planning

$$M_d(t) = 2k_d bte^{-bt^2}$$

$$Y_d(t) = k_d (1 - e^{-bt^2})$$

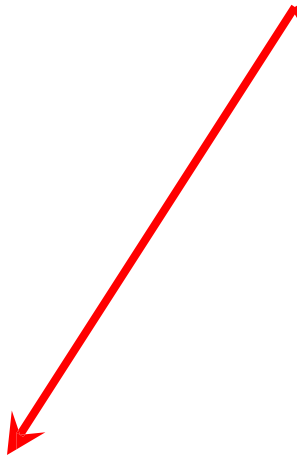
Here $t = t_{od}$

$$\text{Peak manning} = m_{od} = Dt_{od}e^{-1/2}$$

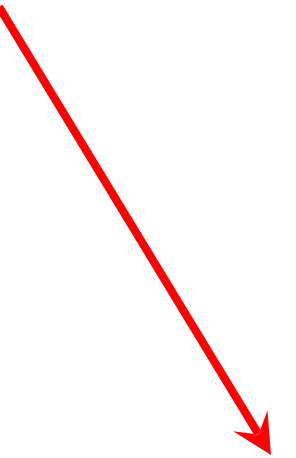
$$= 22.5 \times 1.2 \times .606 = 16 \text{ persons}$$

Software Project Planning

III. If development time is reduced by 2 months



Developing
s/w at higher
manpower
build-up



Producing
less software

Software Project Planning

(i) Increase Manpower Build-up

$$D_o = \frac{1}{t_d^7} \frac{S^3}{C}$$

Now $t_d = 3 \text{ years} - 2 \text{ months} = 2.8 \text{ years}$

$$D_o = (25)^3 / (2.8)^7 = 11.6 \text{ persons / years}$$

$$k = D_o t_d^3 = 254 \text{ PY}$$

$$K_d = \frac{254}{6} = 42.4 \text{ PY}$$

Software Project Planning

$$D = D_0 t_d = 32.5 \text{ persons / year}$$

The peak time is $t_{od} = 1.14 \text{ years}$

$$\begin{aligned} \text{Peak manning } m_{od} &= D t_{od} e^{-0.5} \\ &= 32.5 \times 1.14 \times 0.6 \\ &= 22 \text{ persons} \end{aligned}$$

Note the huge increase in peak manning & manpower cost.

Software Project Planning

(ii) Produce Less Software

$$\frac{S}{C} = D_0 t_d^7 = 7.5 \times (2.8)^7 = 10119.696$$

$$\frac{S}{C} = 21.62989$$

Then for $C=2200$

$S=47586$ LOC

Productivity versus difficult

Example 4.19

A stand alone project for which the size is estimated at 12500 LOC is to be developed in an environment such that the technology factor is 1200. Choosing a manpower build up $D_o=15$, Calculate the minimum development time, total development man power cost, the difficulty, the peak manning, the development peak time, and the development productivity.

Software Project Planning

Solution

Size (S) = 12500 LOC

Technology factor (C) = 1200

Manpower buildup (D_o) = 15

Now $S = CK^{1/3}t_d^{4/3}$

$$\frac{S}{C} = K^{1/3}t_d^{4/3}$$

$$\left(\frac{S}{C}\right)^3 = Kt_d^4$$

Software Project Planning

Also we know $D_o = \frac{K}{t_d^3}$

$$K = D_o t_d^3 = D_o t_d^3$$

Hence $\frac{S}{C} = D_o t_d^7$

Substituting the values, we get $\frac{2500}{1200} = 15 t_d^7$

$$t_d = \frac{(10.416)^{1/7}}{15}$$

$$t_d = 1.85 \text{ years}$$

Software Project Planning

(i) Hence Minimum development time (t_d)=1.85 years

(ii) Total development manpower cost $K_d = \frac{K}{6}$

$$\text{Hence } K = 15t_d^3$$

$$= 15(1.85)^3 = 94.97 \text{ PY}$$

$$K_d = \frac{K}{6} = \frac{94.97}{6} = 15.83 \text{ PY}$$

(iii) Difficulty $D = \frac{K}{t_d^2} = \frac{94.97}{(1.85)^2} = 27.75 \text{ Persons / year}$

Software Project Planning

(iv) Peak Manning $m_0 = \frac{K}{t_d \sqrt{e}}$

$$= \frac{9497}{1.85 \times 1.648} = 31.15 \text{ Person}$$

(v) Development Peak time $t_{od} = \frac{t_d}{\sqrt{6}}$

$$= \frac{1.85}{2.449} = 0.755 \text{ years}$$

Software Project Planning

(vi) Development Productivity

$$= \frac{\text{No .of lines of code (} S \text{)}}{\text{effort (} K_d \text{)}}$$

$$= \frac{12500}{15.83} = 789.6 \text{ LOC / PY}$$

Software Project Planning

Software Risk Management

- We Software developers are extremely optimists.
- We assume, everything will go exactly as planned.
- Other view
 - ↓
not possible to predict what is going to happen ?
 - Software surprises
 - ↓
Never good news

Software Project Planning

Risk management is required to reduce this surprise factor

Dealing with concern before it becomes a crisis.

Quantify probability of failure & consequences of failure.

Software Project Planning

What is risk ?

Tomorrow's problems are today's risks.

“Risk is a problem that may cause some loss or threaten the success of the project, but which has not happened yet”.

Software Project Planning

Risk management is the process of identifying addressing and eliminating these problems before they can damage the project.

Current problems &



Software Project Planning

Typical Software Risk

Capers Jones has identified the top five risk factors that threaten projects in different applications.

1. Dependencies on outside agencies or factors.
 - Availability of trained, experienced persons
 - Inter group dependencies
 - Customer-Furnished items or information
 - Internal & external subcontractor relationships

Software Project Planning

2. Requirement issues

Uncertain requirements



Wrong product

or

Right product badly

Either situation results in unpleasant surprises and unhappy customers.

Software Project Planning

- Lack of clear product vision
- Lack of agreement on product requirements
- Unprioritized requirements
- New market with uncertain needs
- Rapidly changing requirements
- Inadequate Impact analysis of requirements changes

Software Project Planning

3. Management Issues

Project managers usually write the risk management plans, and most people do not wish to air their weaknesses in public.

- Inadequate planning
- Inadequate visibility into actual project status
- Unclear project ownership and decision making
- Staff personality conflicts
- Unrealistic expectation
- Poor communication

Software Project Planning

4. Lack of knowledge

- Inadequate training
- Poor understanding of methods, tools, and techniques
- Inadequate application domain experience
- New Technologies
- Ineffective, poorly documented or neglected processes

Software Project Planning

5. Other risk categories

- Unavailability of adequate testing facilities
- Turnover of essential personnel
- Unachievable performance requirements
- Technical approaches that may not work

Software Project Planning

Risk Management Activities

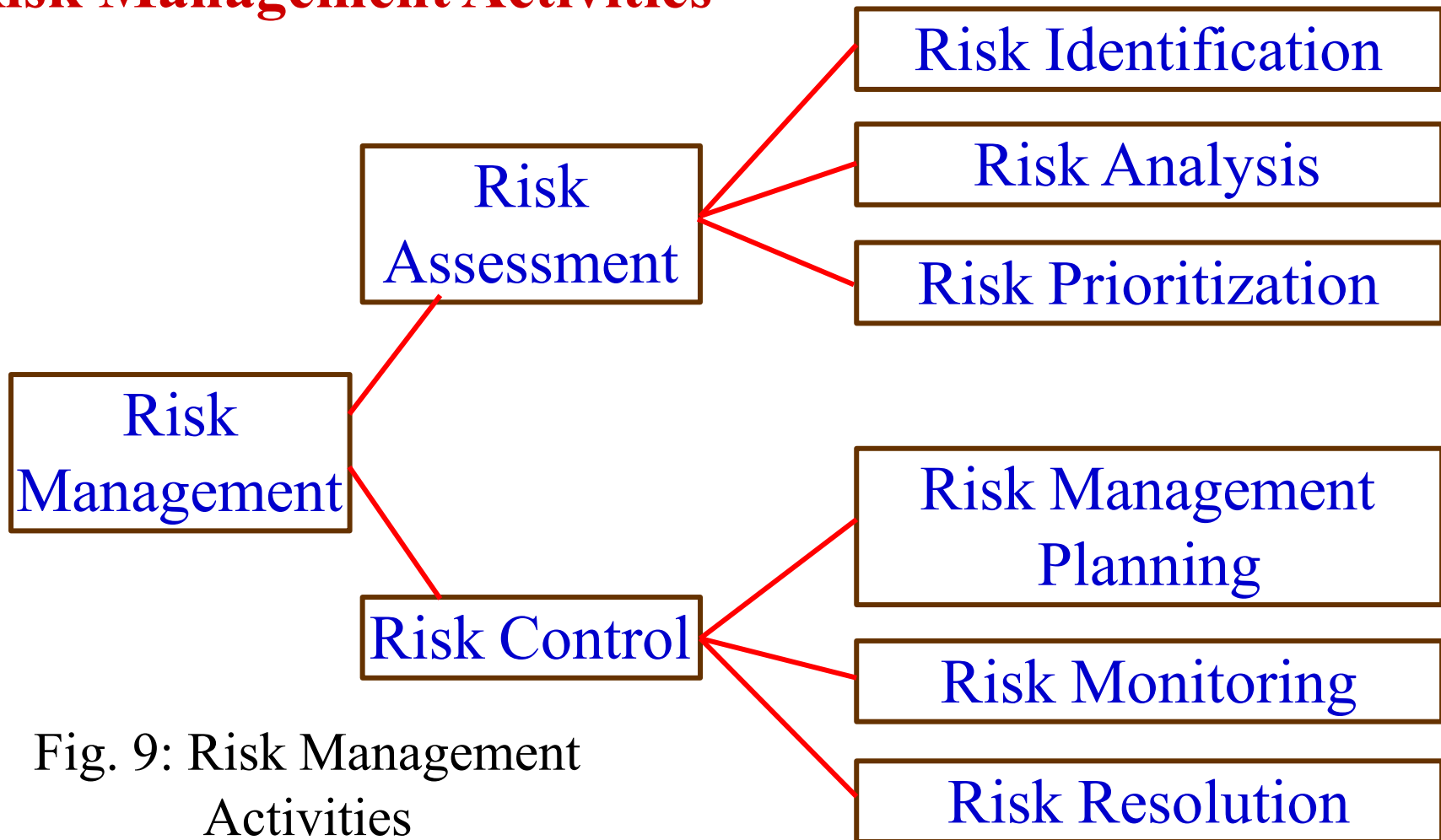


Fig. 9: Risk Management Activities

Software Project Planning

Risk Assessment

Identification of risks

Risk analysis involves examining how project outcomes might change with modification of risk input variables.

Risk prioritization focus for severe risks.

Risk exposure: It is the product of the probability of incurring a loss due to the risk and the potential magnitude of that loss.

Software Project Planning

Another way of handling risk is the risk avoidance. Do not do the risky things! We may avoid risks by not undertaking certain projects, or by relying on proven rather than cutting edge technologies.

Software Project Planning

Risk Control

Risk Management Planning produces a plan for dealing with each significant risks.

- Record decision in the plan.

Risk resolution is the execution of the plans of dealing with each risk.